

Guaranteeing a Dishonest Party’s Knowledge (Or: Setup Requirements for Deniable Authentication)

Nils Fleischhacker  and Guilherme Rito 

Ruhr-Universität Bochum, Germany
{nils.fleischhacker, guilherme.teixeirarito}@rub.de

Abstract. Deniable authentication is a very desirable guarantee for secure messaging — and in particular Off-The-Record messaging: it allows a sender Alice to authenticate her messages while retaining plausible deniability on whether she authenticated any of her messages.

While deniable authentication is not achievable in standard *Public-Key Infrastructures*, Dodis et al. (TCC ’09) have shown that it is achievable in so-called *Key-Registration with Knowledge* setups. These guarantee:

1. dishonest parties’ key-pairs are honestly generated; and
2. dishonest parties have access to their secret keys.

However, these are expensive assumptions, and it is unclear how to securely realize them in practice.

In this paper we study the weakest setup requirements that are sufficient to enable deniable authentication. One of our main findings is that deniable authentication does require an expensive — *but not necessarily impractical* — Key-Registration procedure. Roughly, this procedure must guarantee that if a user’s key registration is successful, then the user can extract *by themselves* valid secret keys from their interaction with the registration authority. We show this setup is *sufficient* by capturing it via an appropriate security model and then proving the anonymity of a ring signature based on it. On the other hand, we explain why this setup *seems inherently necessary* by listing a series of attacks that void deniability as soon as a user manages to successfully register a public key for which it can convincingly claim it does not know a valid (corresponding) secret key.

The requirements we identify for key-registration protocols are expensive, but *not necessarily impractical*. Our second main contribution is showing how to amortize the per-user Key-Registration cost, which brings deniable authentication guarantees a step closer to practicality.

1 Introduction

Deniable authentication is a very desirable for secure communication: it allows a sender Alice to authenticate a message m while retaining plausible deniability on whether she authenticated m [28, 39]. This combination of authenticity and plausible deniability has various applications, most prominently Off-The-Record messaging [17, 18, 20, 36]. Naturally there is a rather vast literature on this topic [18, 20, 23, 24, 28, 33, 35, 36, 37, 38, 39, 45]. As we now explain, this entire literature relies on rather strong setup assumptions, and it is not clear whether such setups can be obtained in practice.

Public-Key Infrastructures. It is well known that deniable authentication guarantees are not achievable in standard Public-Key Infrastructures (PKI) because dishonest users can register public keys without knowing the corresponding secret-keys. For instance, a dishonest Bob could cooperate with a dishonest Judy by having Judy sample a key-pair $(\mathbf{pk}_{\text{Judy}}, \mathbf{sk}_{\text{Judy}})$ honestly which Bob would then register as his own; during registration, Bob would forward messages back and forth between the registration authority and Judy.

Key-Registration with Knowledge. In [23] Dodis et al. introduced a setup called “Key-Registration with Knowledge”. This setup guarantees 1. that every party’s key-pair is sampled honestly — including the key-pairs of dishonest parties — and 2. that every party — crucially including dishonest ones — has access to their secret key (and therefore can use it).¹ Since their seminal result, security models for schemes providing plausible deniable guarantees have been assuming — in one way or another — setups that provide the same guarantees as “Key-Registration with Knowledge” [13, 20, 18, 35, 36, 25]. Yet, it is unclear how this setup could be made practical, which leads us to the problem we study in this paper.

Problem: *What is the weakest setup deniable authentication can be based on?* We make very significant progress in understanding the setup requirements for deniable authentication, and thus in answering this question.

Contribution I. *Key-Registration Requirements:* Our main contribution is introducing Registrable One-Way Relations (Section 5): a formalization of the requirements that key-registration protocols must fulfill to enable deniable authentication. Roughly, whenever a user’s key-registration of a public key $\mathbf{pk}$ is successful, it guarantees that the user can extract a valid secret key $\mathbf{sk}$ corresponding to (the registered) $\mathbf{pk}$ *efficiently, by themselves*.²

This requirement makes the key-registration procedure expensive: if a user’s registration succeeds then it must be ensured that the user did not communicate with any external entity during the registration procedure. This user isolation during registration is necessary to guarantee that users who register really know — and can make use of — their secret keys. (The PKI attack explained above relied exactly on a user not knowing its secret key, thanks to interaction during the key-registration procedure).

Sufficient for Deniable Authentication: We show that our registration requirements are sufficient to enable deniable authentication. We do this by introducing

¹ Dodis et al. refer to the setup they introduce — “Key-Registration with Knowledge” — as a Public-Key Infrastructure [23]. But to the best of our knowledge, the protocol that certification authorities follow does not provide either guarantee. The official guidelines do include an empty section on “Method to Prove Possession of Private Key” [2, Section 3.2.1]. The guidelines published, e.g., by *DigiCert, Inc.* ([4]), *Apple, Inc.* ([3]) and *Microsoft Corporation* ([1]) are prone to the attack explained above for the case of a PKI.

² Crucially, protocols for which this secret key extraction crucially relies on some trapdoor that users do not possess cannot be proven to satisfy our security models.

the *first security model* that appropriately captures these requirements (Sections 5 and 6). Then we show how to construct a ring signature — the simplest technical formalization of deniable authentication — that we prove to be anonymous under this model (Section 6).

“Necessary” for Deniable Authentication: We also explain why these registration requirements seem necessary. We do this by providing a systematized list of attacks that void plausible deniability guarantees as soon as a user manages to successfully register a public key for which it plausibly does not know a corresponding valid secret key. (These attacks are briefly referred to in Section 2; they are explicitly described in the Appendix Section E).

Contribution II. Amortizing Per-User Key-Registration Costs: Naturally, a user may rely on digital computing devices to communicate in a secure manner, for instance “over the internet”. This means that their secret keys have to be somehow be stored in these digital devices.

*What if a user’s device gets compromised?*³ When a user’s device gets compromised, the user has to ensure their public key is invalidated (e.g. by publicly declaring it as stolen): otherwise the attacker who compromised this user’s device would be able to impersonate the user. But this means the user has to undergo the registration procedure once again to register a new, uncompromised, key-pair (or otherwise they can no longer communicate securely). This is very undesirable given the requirements of user registration. Our second main contribution is showing that *it is possible to amortize the key-registration costs per-user*, so that each user only has to register at most one public key. Roughly, we achieve this by providing a mechanism to “extend” registered public keys and corresponding secret keys.

This extension mechanism allows a user Bob — who has already successfully registered key-pair (pk_{Bob}, sk_{Bob}) with a registration authority — to pick an arbitrary extension string “phone” and use it to obtain an extended secret key $sk_{Bob}^{“phone”}$ which he stores in his device; this key allows his phone device to generate signatures with respect to $pk_{Bob}^{“phone”}$ — which is the extension of his public key pk_{Bob} with respect to string “phone”. If Bob’s phone gets compromised, it is guaranteed that even if the attacker obtains extended secret key $sk_{Bob}^{“phone”}$, it cannot obtain the original sk_{Bob} . On the other hand, Bob can simply discard sk_{Bob}^s and obtain a new independent extension of sk_{Bob} . As for deniability guarantees, it is guaranteed that Bob *can always obtain* extensions of his secret key for any arbitrary extension string. This guarantees that even a dishonest Bob cannot find an extension string s with respect to which it cannot sign.

This setup with extendable key-pairs is formalized as Transitive Registrable One-Way Relations (Section 7); we give a more concrete example of how to use it in Section 2.5.

Disclaimer: Our work is concurrent to [24].

³ Users’ device compromises are not an unorthodox attack vector. In fact, this is the reason why current secure messaging applications aim at providing forward-secrecy and post-compromise security guarantees [46, 30, 6, 47, 7, 21, 8, 9, 32].

2 Technical Overview

There are several different primitives and protocols that aim at guaranteeing deniable authentication. While the basic ideas we develop are applicable to many of these, we mainly focus on ring signatures.

In a ring signature scheme the signer can choose an arbitrary set of (valid) public keys and sign under this so called “ring”. Deniability in this context means that it should be impossible to decide which of the ring members actually produced the signature. This should in particular still hold, even if members of the ring collude with the distinguisher. I.e., a ring member should not be able to convince anyone that they did *not* produce a signature. The usual definition of anonymity only considers parties that honestly generate their public key. Therefore the anonymity guarantee may no longer hold once parties can generate their key arbitrarily. Consider for example a ring signature scheme where public keys are group elements, then a malicious party might produce their public key by hashing some public string into the group (as in [28, Section 7, First enumerate, Point 1.]). If we model the hash function as a random oracle, then this public key would be uniformly distributed and thus “look valid”. However, if the dlog problem in the group is hard, the malicious party can prove that they are unlikely to know the secret key – and thus unlikely to be able to sign – by presenting the preimage of the key under the hash function. More generally, if the ring signature scheme has the property that public keys can be sampled obliviously, i.e. without knowledge of the secret key, then anonymity is not guaranteed against malicious keys, even if the signer tries to verify in some way that keys are “valid”.

2.1 Modeling Deniable Authentication in the Presence of Malicious Keys

Intuitively, we want to require is that there is some notion of “validity” such that the following holds:

Whoever presents a “valid” key is capable of signing under a ring containing it.

Formalizing this turns out to be highly non-trivial. Intuitively it might seem simple: just require that any public key has to include a non-interactive proof guaranteeing that the key was honestly generated. However, this is not good enough. A non-interactive proof is always transferrable. This means that a malicious party could simply copy and publish an honest party’s public key as their own. Not only would they now have a public key that they almost certainly could not sign under, they could also convincingly argue that this is the case by showing who’s key they copied.

We must therefore guarantee that the owner of a public key must prove that they can *use* their public key in some *non-transferrable* way. This implies an interactive proof.

Of course a signer cannot require the members of the ring to engage in such a proof with them, not least of all because in many scenarios the ring members

may not actually be willing to cooperate. This means that some kind of key registration must happen with some more or less centralized authority.

We thus model key registration as an interactive protocol between a party and a registration authority, with the guarantee being that anyone who successfully registers a public key with an honest registration authority should be capable of producing signatures on arbitrary messages and for arbitrary rings containing the registered public key. To model this guarantee, we require the ring signature scheme to possess an explicit signature extraction algorithm, that is guaranteed to produce valid signatures on arbitrary messages under arbitrary rings, given only the view, i.e. input, randomness, and transcript of the registering party. Note, that we do *not* require that the *secret key* can be extracted. This would be a strictly stronger and unnecessary requirement, even though it is likely that most constructions of such a scheme would do exactly this. It is important that the extraction procedure is explicitly part of the description of the ring signature scheme and – contrary to e.g. common definitions of proofs of knowledge – makes no use of a trapdoor or other secret input that the real world party registering the key would not have access to. Otherwise we cannot guarantee that the extraction actually works in practice.

If we only required the *existence* of an extractor, this extractor might still be (provably) hard to find. If a trapdoor is required to extract, then the real world party may simply not be able to run the extractor and may be able to prove that they can't.

However, it turns out that just requiring an interactive registration protocol with an authority still *cannot* guarantee that the party registering the key is actually capable of signing. To see this consider the following scenario: Mallory is anticipating that he will at some point in the future need to convince Judy, that he did not issue some particular ring signature. To be able to do this, he asks Judy to generate a key on his behalf and then registers this key by simply forwarding messages between the registration authority and Judy. The registration authority is now convinced that *someone* will be able to sign under the public key. The problem is that this someone isn't Mallory. At any point in the future, Judy will be easily convinced that Mallory did not issue any signature under said public key, because she herself generated it and did not give the secret key to Mallory.

This does mean that the actual registration process in the real world must somehow ensure that Mallory can't communicate with any third party during the registration process. In the real world this can e.g. be achieved by requiring Mallory to present himself at a specific physical location and get into a Faraday cage for the duration of the registration process. Although this is admittedly very costly, it does appear to be necessary considering the attack described above.

In our formal definition of deniability this is modeled by requiring the attacker to specify an arbitrary interactive algorithm that performs the registration and then running the interaction between this algorithm and the registration authority without further input from the attacker.

Trust in the Registration Authority Clearly, in the above scenario, the signer needs to trust the registration authority to correctly perform the registration.

Otherwise we could get no guarantee. We specifically choose to separate key *generation* from key *registration* to allow a party to register the same key with multiple authorities. This allows some level of decentralization, where different parties might trust different registration authorities.

On the other hand, there's no reason that the party registering the key should need to trust the registration authority at all. To model this, the definition of unforgeability explicitly gives the attacker the ability to act as a registration authority and they should still not be capable of forging signatures under honest rings.

2.2 Registrable One-Way Relations

The main building block for our registration-based ring signatures are what we call registrable one-way relations (ROWR). A registrable one-way relation is essentially an NP relation $\mathcal{R}$ that is equipped with a statement registration protocol that satisfies two main security properties.

First it needs to be one-way, in the sense that for honestly sampled statements x even a party impersonating the registration authority is not capable of finding a valid witness w for x . Second it needs to allow witness extraction in the following sense: there exists an explicit extraction algorithm that, given the *view* of the registering party (and nothing else, in particular no trapdoor or similar secret information) is able to extract a valid witness for the registered statement.

The registered statements will be what is used as public keys by our ring signature scheme. This makes our construction very general. In principal almost anything can be used as a public key, as long as there is some efficient way of verifying that the corresponding secret is actually correct. In particular one could even consider a “universal” relation.

$$\mathcal{R}_{\text{uni}}\{((x, C), w) \mid C(x, w) = 1\}$$

where C denotes an arbitrary boolean circuit. A ring signature scheme for this relation, would allow the signer to include completely different *kinds* of keys for every single member of the ring.⁴

The owner of a key would not even have to necessarily “explain” their secret key and could hide the actual verifying circuit. Say a user wants to use a statement/witness pair of relation $\mathcal{R}' = \{(x, w) \mid C'(x, w) = 1\}$ for some circuit C' but does not want to reveal C' . Using a hiding and binding commitment scheme the user could instead register the statement $((x, c = \text{Comm}(C'; r)), C)$, with witness (C', r, w) where circuit C is defined as $C((x, c), (C', r, w)) := (\text{Comm}(C'; r) = c \wedge C'(x, w) = 1)$.

This means that basing our ring signatures on ROWR enforces very little structure on what can be used as keys within the system.

⁴ We note that, while this is possible in principle, we do not know of a reasonably efficient instantiation.

2.3 Constructing Registration-Based Ring Signatures

In addition to a ROWR the construction makes use of a CPA secure public key encryption scheme, a simulation sound non-interactive zero knowledge proof system, and a hash function. The construction of the ring signature scheme is then relatively straightforward.

- Key generation of the ring signature scheme simply samples from the relation of the ROWR, using the statement x as the public key and the witness w as the secret key.
- The key registration protocol is the statement registration protocol of the ROWR.
- To sign a message m under a ring $(x_1, \dots, x_\ell)$, encrypt the witness w as well as a hash $h = H(m)$ of the message and prove in zero-knowledge that the ciphertext encrypts $H(m)$ as well as a witness, such that for some $1 \leq i \leq \ell$, $(x_i, w) \in \mathcal{R}$.
- To verify simply recompute $h = H(m)$ and verify the NIZK.

Intuitively, the security of the scheme is relatively easy to see, even if the proofs are somewhat involved.

Unforgeability Unforgeability essentially follows from the simulation soundness of the NIZK and the one-wayness of the ROWR (as well as collision resistance of the hash function). Any forgery must include a valid NIZK asserting that the ciphertext contains a valid witness for one of the honestly registered keys in the ring. By the soundness of the NIZK and the correctness of the encryption scheme, we can thus decrypt this witness and thus break the one-wayness of the ROWR.

Deniability By the witness extractability of the ROWR, we know that any party who registered a statement, can in fact extract a witness for that statement and then simply use the regular signing algorithm to produce a valid signature.

2.4 Amortizing Registration Cost

We previously noted that key registration needs to essentially take place *in person* in some specially designed facility. While this seems strictly *necessary*, it is an unfortunate and extreme cost to bear for the user.

The cost becomes even worse when one considers rotating keys. To actually make use of their key the user almost certainly has to store it in something like their mobile device that might at some point get either lost or compromised. Even if nothing bad happens, users might want to rotate their key every so often just to be sure. If the user had to reregister their key in person every single time, that would be a huge additional cost to bear. One idea to amortize the cost of key registration is to allow the user to register a public key *once* and store the corresponding secret key in a secure location, i.e. a secure device not connected to any network and kept in a secure physical location, but never use it directly.

Instead whenever the user needs a new key, they should be able to *derive* a new key public key that is verifiably related to the registered key, in such a way that the deniability guarantee is transferred to these derived keys. The derived key can then be stored in e.g. a mobile device. We can do this by replacing the Registrable One-Way Relation in the above construction with what we call a Transitive Registrable One-Way Relation.

2.5 Transitive Registrable One-Way Relations

A Transitive Registrable One-Way Relation works exactly like a Registrable One-Way Relation but has some additional properties: Given a statement x and an *extension* e , an extended statement x_e can be computed. Similarly, given a witness w for x and an *extension* e , an extended witness w_e for x_e can be computed.

First a Transitive Registrable One-Way Relation still needs to be one-way. In this case that means that even given access to extended witnesses for arbitrary extensions, a party potentially impersonating the registration authority should not be capable of finding a valid witness w for x . Second it also requires a stronger notion of witness extraction: there needs to exist an explicit extraction algorithm that, given the *view* of the registering party (and nothing else, in particular no trapdoor or similar secret information) is able to extract a valid extended witness for any extension of the registered statement.

Conceptually we can simply replace the ROWR in the above construction of registration based ring signatures with a TROWR to achieve low amortized registration cost.

3 Related Work

The literature on deniable authentication is vast and dates back to the seminal work by Jakobsson, Sako and Impagliazzo, who introduced Designated Verifier Proofs (DVP) and Signatures (DVS) [28]. Their work already makes mention (though rather informally) of the base requirements that are necessary to enable the guarantees provided by DVS and DVP. For instance, their threat model makes the following assumption:

“the verifier has not totally lost control of his ability to access his data, perform calculations and freely communicate.” ([28, “2. Approach and Model”])

Interestingly, some of the attacks that motivate our setup had already been identified by Jakobsson, Sako and Impagliazzo in [28, “7. Practical Issues”], though informally. They also explain how these attacks translate into setup requirements that are necessary to enable plausible deniability guarantees — in particular, by ensuring the setup prevents these attacks:

[28, “7. Practical Issues”, Point 1 of second enumerate] a verifier must prove knowledge of their secret key;

[28, “7. Practical Issues”, Point 1 of third enumerate] a verifier must undergo a Key-Registration during which they cannot communicate with outsiders.

However, as discussed in Section 2, we do not know how to prove that (an adequate formalization of) these requirements are sufficient. This is because it is not ruled out that a verifier can successfully register a public key without actually being able to extract the corresponding secret key by themselves.

In [45], Steinfeld et al. introduce Universal Designated Verifier Signatures (UDVS). These schemes are a variant of Designated Verifier Signatures wherein a signer first produces a standard, publicly verifiable signature, and then anyone can take such signature and designate it to a particular receiver (hence the term universal). Interestingly, the UDVS security model that Steinfeld et al. introduce explicitly includes a “Key-Registration Protocol” that verifiers should use to register their keys.

Rivest, Shamir and Tauman introduce Ring Signatures [39], which rely on essentially the same setup as Designated Verifier Signatures. ([39] does not make the assumptions on their setup explicit.)

The work that is most closely related to ours is by Ristenpart and Yilek, who introduce Proofs-of-Possession [38]. The main difference between our works, which is actually the *crucial feature* of our security model, is that their key-registration protocols do not capture that if a user manages to register then the user can extract his own secret key by themselves. This is necessary because without this guarantee we cannot rule out that the user could potentially not actually know how to use their secret key, which would preclude plausible deniability guarantees.

Comparison with Concurrent Work [24] In concurrent work Fiedler and Langrehr [24] model deniable authentication in the presence of maliciously chosen public keys for Designated Verifier Signatures (DVS). DVS are a slightly simpler setting, because there are only ever two parties involved and only the designated signer is “normally” supposed to sign. Specifically Fiedler and Langrehr define *simplified malicious source hiding* to capture deniable authentication in the presence of maliciously chosen verifier keys.

Abstracting from the differences between DVS and ring signatures, their notion and ours are broadly similar, however there are some key differences. The most important one is that their notion of a DVS does *not* include a mechanism of key registration. It turns out that this is in fact a problem with their notion. In [24] they prove under standard assumptions that an unforgeable simplified malicious source hiding DVS cannot be instantiated in the standard model.⁵ The reason is essentially that a party might still — as discussed above — have

⁵ [24] includes an instantiation in the ROM. However their impossibility result is an explicit attack on any standard model instantiation. Therefore this includes the ROM construction once the RO is instantiated with any efficiently computable function. Thus, this is more an indictment of the ROM than a usable construction.

the ability to *obviously* sample public keys. In their impossibility proof this is generically achieved by giving the adversary access to an obfuscated version of the key generation algorithm.

We note that this generic attack cannot be extended to our definition. Even if an adversary can obviously sample public keys, they would not generically be able to *register* this key, as registration uses the secret key.

In a minor difference, Fiedler and Langrehr choose to give the adversary against simplified malicious source hiding (as well as the extractor) access to an explicit auxiliary input to model external data (such as pre-existing keys from elsewhere) being used by the adversary. Our definition does *not* have such an explicit auxiliary input, however, neither is it necessary. All our definitions are statements of *finite security* [12, 35]. In particular this means that our security is defined for an a priori *fixed* security parameter and therefore the adversary's code can depend on the security parameter. This means that whatever one might want to pass to the adversary as an auxiliary input — even if it depends on the security parameter — can simply be hardcoded into the description of the adversary.

Extractable One-way Functions. In [14], Bitansky et al. introduce Extractable One-Way Functions — which are similar to Registrable One-Way Relation — and prove the impossibility of constructing them in the standard model. However, we note that their impossibility result also stems from the fact that their security model does not include a key-registration procedure, leading to problems that are analogous to the ones in [24].

4 Preliminaries

We denote the arity of a vector $\vec{x}$ by $|\vec{x}|$ and its i -th element by x_i . We write $\alpha \in \vec{x}$ to denote $\exists i \in \{1, \dots, |\vec{x}|\}$ with $\alpha = x_i$. We write $\text{Set}(\vec{x})$ to denote the set induced by vector $\vec{x}$, i.e. $\text{Set}(\vec{x}) := \{x_i \mid x_i \in \vec{x}\}$. For a binary relation R we denote by L_R the language induced by R , i.e. $L_R := \{x \mid \exists w, (x, w) \in R\}$. For a set S , we denote the set of finite strings over S by S^* and call S an alphabet. In particular, the empty string, denoted ε , is an element of S^* . The set of strings over S of a given length $l \in \mathbb{N}$ is denoted S^l , and the set of non-empty strings over S is denoted S^+ . We denote the concatenation of strings u and v by $u \parallel v$. For a string m , we will denote by $m[i..j]$ the substring of m starting at its i -th symbol and up to (including) its j -th symbol. Furthermore, for a vector of strings $\vec{m}$, we denote by $\vec{m}[1..j]$ the concatenation of strings m_1 up to m_j , i.e. $\vec{m}[1..j] = m_1 \parallel \dots \parallel m_j$.

Vector notation: Throughout the paper we frequently use vectors. We use upper case letters to denote vectors of parties, and lower case letters to denote vectors of artifacts such as public keys, sequences of random coins, etc. Moreover, we use the convention that if $\vec{V}$ is a vector of parties, then $\vec{v}$ denotes $\vec{V}$'s corresponding vector of public keys. For example, for a vector of parties $\vec{V} := (\text{Bob}, \text{Charlie})$,

$\vec{v} := (\mathbf{pk}_{\text{Bob}}, \mathbf{pk}_{\text{Charlie}})$ is $\vec{V}$'s corresponding vector of public keys. In particular, V_1 is Bob and v_1 is Bob's public key $\mathbf{pk}_{\text{Bob}}$, and V_2 is Charlie and v_2 is Charlie's public key $\mathbf{pk}_{\text{Charlie}}$. More generally, for a vector of parties $\vec{V}$ with corresponding vector of public keys $\vec{v}$, V_i 's public key is v_i , for $i \in \{1, \dots, |\vec{V}|\}$.

Algorithm interaction: Interactive algorithms can be defined as Interactive Turing Machines (ITMs). Consider two interactive algorithms A and B. We denote the interaction of A and B, run on input in_A and in_B , respectively, by

$$\langle \text{out}^A \mid \text{out}^B \rangle \leftarrow \langle A(\text{in}_A) \leftrightarrow B(\text{in}_B) \rangle$$

where $\langle \text{out}^A \mid \text{out}^B \rangle$ is the interaction's output: out^A and out^B are, respectively, the outputs of A and B when they halt. We assume A and B's interaction proceeds in rounds, and that in each round only one of them is active and may have an output. If in round i A is active then in round $i + 1$ B will be active (unless it already halted). We distinguish among two types of output: the output of an algorithm when it halts — for algorithm X this is denoted out^X — and the interaction output of an algorithm — corresponding to the message it sends to the other algorithm in that round. The i -th interaction message, denoted m_i , is the one output at round i . In the example above, if A outputs a message m_i at round i , then in round $i + 1$ B will be run on interaction input m_i (if it did not already halt). Let X be the first algorithm (among A and B) that halts and Y the other one, and let i be the round in which it halts. In addition to defining out^X , algorithm X may also output a final interaction message m_i . If no interaction message is output then out^Y is set to special symbol **undef** (denoting the value is undefined) and we say the interaction took i rounds. Otherwise, round $i + 1$ is the last one, and we say the interaction took $i + 1$ rounds. In the latter case, in the last round $i + 1$ algorithm Y is run on input m_i ; if Y halts in that round then it defines its output out^Y , but otherwise it is set to special symbol **undef**. Algorithm Y may still output a final interaction message m_{i+1} that will be visible to external observers of this interaction, though it will not be given to X because it has already halted. If Y does not output a final interaction message, we assume we still define m_{i+1} and set it to **undef**. For an interaction $\langle A(\text{in}_A) \leftrightarrow B(\text{in}_B) \rangle$ that takes r rounds, and letting $\vec{m} := (m_1, \dots, m_r)$ be the of interactive messages exchanged between A and B, we denote this interaction's transcript $\vec{m}$ by

$$\text{Transc}_{\langle A(\text{in}_A) \leftrightarrow B(\text{in}_B) \rangle}.$$

We follow the convention that the left-hand side algorithm always initiates the interaction (by sending the first message); in the example above, this means A sends the first message to B in their interaction. For $X \in \{A, B\}$, we denote the i -th message output by X in the interaction above by m_i^X .

Finally, in our work we rely on the abilities to both (re-)run algorithms given a non-halted configuration of theirs, and store an algorithm's configuration at some point in their execution (which may be later used to rerun the algorithm from a past configuration).

Game-based security definitions: We follow [29] in defining game-based notions, i.e. we define them via a set of oracles that adversaries can interact with. We denote an oracle $\mathbf{X}$ by $\mathcal{O}[\mathbf{X}]$, and oracles $\mathbf{X}$ and $\mathbf{Y}$ by $\vec{\mathcal{O}}[\mathbf{X}, \mathbf{Y}]$. We differentiate two types of notions: *event-based* and *distinction-based*.

Event-based notions. Event-based security notions capture adversaries' abilities of causing some "bad" event ξ to occur — as defined by the notion — when interacting with a set of oracles, say $\vec{\mathcal{O}}[\mathbf{X}_1, \dots, \mathbf{X}_n]$. These notions may also specify what oracle queries adversaries are not allowed to make. We say an adversary $\mathbf{A}$ triggers event ξ if $\mathbf{A}$ does not make any disallowed queries and ξ does occur; we denote $\mathbf{A}$'s probability of triggering ξ by $\Pr[\mathbf{A}^{\vec{\mathcal{O}}[\mathbf{X}_1, \dots, \mathbf{X}_n]} \Rightarrow \xi]$ and define $\mathbf{A}$'s advantage as $Adv_{\vec{\mathcal{O}}[\mathbf{X}_1, \dots, \mathbf{X}_n]}^\xi(\mathbf{A}) := \Pr[\mathbf{A}^{\vec{\mathcal{O}}[\mathbf{X}_1, \dots, \mathbf{X}_n]} \Rightarrow \xi]$. When clear from context, we omit the oracles, writing instead $Adv^\xi(\mathbf{A}) := \Pr[\mathbf{A} \Rightarrow \xi]$. Similarly, we will refer to the list of oracles $\vec{\mathcal{O}}[\mathbf{X}_1, \dots, \mathbf{X}_n]$ with respect to which an event ξ is defined by $\mathbf{G}^\xi$, and will denote an adversary $\mathbf{A}$'s interaction with these oracles by $\mathbf{A} \cdot \mathbf{G}^\xi$ rather than $\mathbf{A}^{\vec{\mathcal{O}}[\mathbf{X}_1, \dots, \mathbf{X}_n]}$.

For two event-based notions providing adversaries access to the same oracles that capture adversaries' ability to trigger events ξ_1 and ξ_2 , one can define a combined notion capturing adversaries' ability to trigger ξ_1 or ξ_2 ; we denote an adversary $\mathbf{A}$'s winning probability as $\Pr[\mathbf{A} \Rightarrow \xi_1 \vee \xi_2]$ and denote the advantage by $Adv^{\xi_1 \vee \xi_2}(\mathbf{A}) := \Pr[\mathbf{A} \Rightarrow \xi_1 \vee \xi_2]$.

Distinction-based notions. For this type of notions the goal of an adversary $\mathbf{A}$ is correctly guessing a challenge bit $\mathbf{b} \in \{0, 1\}$. For example, one can define a notion IND by defining two sets of oracles $\vec{\mathcal{O}}_{\text{IND}}^{\mathbf{b}} := \vec{\mathcal{O}}[\mathbf{X}_1^{\mathbf{b}}, \dots, \mathbf{X}_n^{\mathbf{b}}]$, for $\mathbf{b} \in \{0, 1\}$; an adversary $\mathbf{A}$ interacts with either $\vec{\mathcal{O}}_{\text{IND}}^0$ or $\vec{\mathcal{O}}_{\text{IND}}^1$, and at some point outputs a guess bit $b_{\mathbf{A}} \in \{0, 1\}$; generally, for $\mathbf{b} \in \{0, 1\}$, $\mathbf{A}$ wins (when interacting with $\vec{\mathcal{O}}_{\text{IND}}^{\mathbf{b}}$ if it guesses $\mathbf{b}$ correctly (i.e. if $b_{\mathbf{A}} = \mathbf{b}$); we denote this event by **win**. Security notions may disallow certain oracle queries; if an adversary makes a disallowed query, the bit it outputs as its guess is replaced with one sampled uniformly at random. For $\mathbf{b} \in \{0, 1\}$, we denote $\mathbf{A}$'s winning probability against IND's $\mathbf{b}$ game by $\Pr[\mathbf{A}^{\vec{\mathcal{O}}_{\text{IND}}^{\mathbf{b}}} \Rightarrow \text{win}]$, and define $\mathbf{A}$'s advantage as

$$Adv^{\text{IND}}(\mathbf{A}) := \left| \Pr[\mathbf{A}^{\vec{\mathcal{O}}_{\text{IND}}^0} \Rightarrow \text{win}] + \Pr[\mathbf{A}^{\vec{\mathcal{O}}_{\text{IND}}^1} \Rightarrow \text{win}] - 1 \right|.$$

For $\mathbf{b} \in \{0, 1\}$, we may alternatively denote by $\mathbf{G}_{\mathbf{b}}^{\text{IND}}$ oracle set $\vec{\mathcal{O}}_{\text{IND}}^{\mathbf{b}}$, and denote an adversary $\mathbf{A}$'s interaction with these oracles by $\mathbf{A}\mathbf{G}_{\mathbf{b}}^{\text{IND}}$ rather than $\mathbf{A}^{\vec{\mathcal{O}}_{\text{IND}}^{\mathbf{b}}}$.

5 Registrable One-Way Relations

This section formalizes and explains how to instantiate Registrable One-Way Relations. To smoothen the introduction of these schemes, we first present One-Way Relations, a binary relation variant of One-Way Functions.

5.1 One-Way Relations

Consider some binary relation $R \subseteq \mathcal{X} \times \mathcal{W}$. We refer to elements of $\mathcal{X}$ and $\mathcal{W}$ as statements and witnesses, respectively; if $(x, w) \in R$, we call w a witness for x .

A One-Way Relation (OWR) for a relation $R \subseteq \mathcal{X} \times \mathcal{W}$ is a pair $\text{OWR} = (\text{Samp}, D_{\in R})$, where Samp and $D_{\in R}$ are, respectively, probabilistic and deterministic algorithms:

- Samp : outputs a pair $(x, w) \in \mathcal{X} \times \mathcal{W}$;
- $D_{\in R}(x, w)$: outputs indicator bit for $(x, w) \in R$.

For simplicity, we will write $(x, w) \in R$ to denote $D_{\in R}(x, w)$ when clear from context.⁶ The One-Wayness notion (Definition 1) relies on the following oracles:

$\mathcal{O}[\text{Witness}](i \in \mathbb{N})$: First call on i : sample $(x_i, w_i) \leftarrow \text{Samp}$. Output (x_i, w_i) .
 $\mathcal{O}[\text{Samp}](i \in \mathbb{N})$: Let $(x_i, w_i) \leftarrow \mathcal{O}[\text{Witness}](i)$. Output x_i .
 $\mathcal{O}[\in](i \in \mathbb{N}, w)$: Let $(x_i, w_i) \leftarrow \mathcal{O}[\text{Witness}](i)$. Output $(x_i, w) \in R$.

The notion ahead captures adaptive security as adversaries can obtain witnesses for any statements they see via oracle $\mathcal{O}[\text{Witness}]$. One can define a (weaker) variant corresponding to static security by not providing adversaries with access to oracle $\mathcal{O}[\text{Witness}]$ (or, equivalently, by making adversaries abort upon the first query to $\mathcal{O}[\text{Witness}]$). We add a superscript **stat** to the notion capturing static security, and none for notions capturing adaptive security.

Definition 1 (One-Wayness: $\neg\text{OW}$). A query $\mathcal{O}[\in](i, w)$ outputs 1. There was no prior query $\mathcal{O}[\text{Witness}](i)$.

Adversary $\mathbf{A}$ (ε, t) -breaks OWR's (n) -Adaptive-One-Wayness if it runs in time t , queries oracles $\mathcal{O}[\text{Samp}]$, $\mathcal{O}[\text{Witness}]$ and $\mathcal{O}[\in]$ on at most n different indices, and satisfies $\text{Adv}^{\text{OW}}(\mathbf{A}) \geq \varepsilon$. We say OWR is (ε, t, n) -Adaptively secure if there is no such adversary. Similarly, $\mathbf{A}$ (ε, t) -breaks OWR's (n) -Static-One-Wayness if it (ε, t) -breaks OWR's (n) -Adaptive-One-Wayness and $\mathbf{A}$ does not query $\mathcal{O}[\text{Witness}]$, and we say that OWR is (ε, t, n) -Statically secure if there is no such adversary. We establish two straightforward facts about One-Way Relations.

Proposition 1 (One-Way Relations $\Leftrightarrow$ One-Way Functions). There is a (ε, t, n) -Statically secure One-Way Relation if and only if there is a (ε, t', n) -secure One-Way Function with $t \approx t'$.

Proposition 2 (Static Security $\Leftrightarrow$ Adaptive Security). If there is a (ε, t, n) -Statically-secure One-Way Relation then there is a (ε', t', n') -Adaptively secure One-Way Relation with $\varepsilon = 2\varepsilon'$, $t \approx t'$ and $n = n'$.

⁶ Alternatively, one could define relation R based on the output of $D_{\in R}$.

5.2 Registrable One-Way Relations

Registrable One-Way Relations are essentially One-Way Relations with an additional registration functionality that allows parties to register statement-witness pairs. More formally, a Registrable One-Way Relation (ROWR) for binary relation $R \subseteq \mathcal{X} \times \mathcal{W}$ is a tuple $\text{ROWR} = (\text{Setup}, \text{Samp}, D_{\in R}, \text{Reg} := (\text{RU}, \text{RA}, \text{Extract}))$, where:

- **Setup**: outputs public parameters pp ;
- **Samp**(pp): outputs a pair $(x, w) \in R$;
- $D_{\in R}(\text{pp}, x, w)$: outputs whether $(x, w) \in R$; and
- **Reg** := $(\text{RU}, \text{RA}, \text{Extract})$ is a triple with
 1. (RU, RA) being interacting algorithms

$$\langle b | x \rangle \leftarrow \langle \text{RU}(\text{pp}, x, w) \leftrightarrow \text{RA}(\text{pp}) \rangle,$$

where RU 's output is a bit b indicating if registration succeeded, and RA 's output is either x (RU 's input) or $\perp$ if registration fails.

2. **Extract**($\text{pp}, U, x \in \mathcal{X}, \text{Transc}_{\langle U \leftrightarrow \text{RA} \rangle}$): given an interactive algorithm's description U , a statement x and a transcript $\text{Transc}_{\langle U \leftrightarrow \text{RA} \rangle}$, outputs a witness $w \in \mathcal{W}$ or $\perp$ (if extraction fails).

Remark 1. As one may note, algorithm **Extract** is not input the random coins of Reg.RA . This is crucial because we want our notion to guarantee that a user can really extract — without any help — a valid witness from any statement that it can successfully register.

The notions ahead provide adversaries with access to the following oracles:

- $\mathcal{O}[\text{Setup}]$: If pp is undefined, sample $\text{pp} \leftarrow \text{Setup}$; output pp .
- $\mathcal{O}[\text{Samp}](i \in \mathbb{N})$: If x_i is undefined, sample $(x_i, w_i) \leftarrow \text{Samp}(\text{pp})$. Output x_i .
- $\mathcal{O}[\text{Witness}](i \in \mathbb{N})$: If x_i was defined by a query $\mathcal{O}[\text{Samp}](i)$, output (x_i, w_i) . Otherwise output $\perp$.
- $\mathcal{O}[\text{Reg.RA}^H](i \in \mathbb{N})$: If x_i is undefined or was defined by a query to $\mathcal{O}[\text{Reg.RA}]$, output $\perp$. Let $(x_i, w_i) \leftarrow \mathcal{O}[\text{Witness}](i)$ and $\text{pp} \leftarrow \mathcal{O}[\text{Setup}]$. Run $\langle b | x \rangle \leftarrow \langle \text{RU}(\text{pp}, x_i, w_i) \leftrightarrow \text{RA}(\text{pp}) \rangle$. Output (b, x) .
- $\mathcal{O}[\text{Reg.RA}](i \in \mathbb{N}, U)$: If x_i is defined output $\perp$. Else, given the description of (stateful) interactive algorithm U , run $\langle \text{out} | x \rangle \leftarrow \langle U \leftrightarrow \text{RA}(\text{pp}) \rangle$, and let $\text{Transc}_{\langle U \leftrightarrow \text{RA} \rangle}$ be the respective interaction transcript. If Reg.RA halted and output $x \neq \perp$, then define $x_i := x$ and compute $w \leftarrow \text{Reg.Extract}(\text{pp}, U, x, \text{Transc}_{\langle U \leftrightarrow \text{RA} \rangle})$; if $w \neq \perp$ define $w_i := w$. Output $(\text{out}, x, w, \text{Transc}_{\langle U \leftrightarrow \text{RA} \rangle})$.
- $\mathcal{O}[\text{Reg.RU}](i \in \mathbb{N}, \text{sid} \in \mathbb{N}, m)$: If x_i was defined by $\mathcal{O}[\text{Reg.RA}]$, output $\perp$. If there is an already stored configuration $\text{Reg.RU}[i, \text{sid}']$ that is not a halted configuration and $\text{sid} \neq \text{sid}'$, then output $\perp$. Else, let this be the j -th query on (i, sid) that did not output $\perp$ due to the condition above:
 - First query (i.e. $j = 1$): Create a new Reg.RU instance with input (pp, x_i, w_i) . Denote it by $\text{Reg.RU}[i, \text{sid}]$. Run $\text{Reg.RU}[i, \text{sid}]$ until it outputs the first message $m_1^{\text{Reg.RU}[i, \text{sid}]}$ and/or halts with some output $\text{out}^{\text{Reg.RU}[i, \text{sid}]}$. (For $j = 1$ input m is ignored.)

$j > 1$: If $\text{Reg.RU}[i, \text{sid}]$ halted in a prior query output $\perp$. Otherwise, load its state and run it with (interaction) input m until it outputs the $j + 1$ -th message $m_{j+1}^{\text{Reg.RU}[i, \text{sid}]}$ and/or halts with some output $\text{out}^{\text{Reg.RU}[i, \text{sid}]}$. Store $\text{Reg.RU}[i, \text{sid}]$'s configuration. (If it halted, record that.) Output pair $(m_1^{\text{Reg.RU}[i, \text{sid}]}, \text{out}^{\text{Reg.RU}[i, \text{sid}]})$, where $m_1^{\text{Reg.RU}[i, \text{sid}]}$ is the interaction message output by $\text{Reg.RU}[i, \text{sid}]$ or undef if it did not output such message, and $\text{out}^{\text{Reg.RU}[i, \text{sid}]}$ is the halting output of $\text{Reg.RU}[i, \text{sid}]$ or undef if it did not halt.

$\mathcal{O}[\epsilon](i \in \mathbb{N}, w)$: Output $(x_i, w) \in R$.

Remark 2 (Input U of $\mathcal{O}[\text{Reg.RA}]$). Note that, even though U is an interactive algorithm, $\mathcal{O}[\text{Reg.RA}]$ runs U locally. This is crucial to guarantee that U is not merely forwarding messages between Reg.RA and whichever party actually knows the witness to the statement being registered.

Remark 3 ($\mathcal{O}[\text{Reg.RU}]$). The notions ahead provide adversaries with access to a user registration oracle to capture that malicious registration authorities should not be able to learn the witnesses for the statements that honest users register. As one may note, we explicitly disallow that honest users participate in concurrent registrations. This is because the requirements one needs to ensure a user knows a witness to the statement they are registering already implies that honest users cannot participate in concurrent registration sessions.

The notion below allows adversaries to be given oracle access to $\mathcal{O}[\text{Reg.RU}]$.

Definition 2 (Registration-Completeness: $\neg\text{RegComplete}$). A statement x_i is defined by a query $\mathcal{O}[\text{Samp}](i)$ or by a query $\mathcal{O}[\text{Witness}](i)$. A query $\mathcal{O}[\text{Reg.RA}^H](i)$ outputs $\perp$ or a pair $(b, x) \neq (1, x_i)$.

Definition 3 (One-Wayness: $\neg\text{OW}$). A query $\mathcal{O}[\epsilon](i, w)$ outputs 1. The output of every query $\mathcal{O}[\text{Reg.RA}](i, \cdot)$ is of the form $(\cdot, x = \perp, \cdot, \cdot)$. There was no prior query $\mathcal{O}[\text{Witness}](i)$.

Definition 4 (Witness-Extractability: $\neg\text{WE}$). A query $\mathcal{O}[\text{Reg.RA}](i, U)$ outputs $(\cdot, x, w, \cdot) \neq \perp$ with $x \neq \perp$. Query $\mathcal{O}[\epsilon](i, w)$ outputs 0.

(The following property follows trivially from One-Wayness; we only state because it helps simplifying a security proof of our Ring Signature construction.)

Definition 5 (γ -Statement-Spreadness). A scheme $\text{ROWR} = (\text{Setup}, \text{Samp}, D_{\in R}, \text{Reg} := (\text{RU}, \text{RA}, \text{Extract}))$ for a relation $R \subseteq \mathcal{X} \times \mathcal{W}$ is γ -Statement-Spread if $\forall \text{pp} \in \text{Supp}(\text{Setup}) : -\log \max_{x \in \mathcal{X}} \Pr_{r_{\S}}[x = \text{Samp}(\text{pp}; r_{\S})] \geq \gamma$. (This implies $\forall \text{pp} \in \text{Supp}(\text{Setup}), x \in \mathcal{X}$ we have $\Pr_{r_{\S}}[x = \text{Samp}(\text{pp}; r_{\S})] \leq 2^{-\gamma}$.)

An adversary $\mathbf{A}(\epsilon, t)$ -breaks the $(n, q_{\text{RA}}^H, q_{\text{RA}}, q_{\text{RU}})$ -Registration-Completeness (resp. One-Wayness and Witness-Extractability) of scheme ROWR if it queries oracles $\tilde{\mathcal{O}}[\text{SK}, \text{VK}]$ on at most n distinct indices, makes at most $q_{\text{RA}}^H, q_{\text{RA}}$ and

q_{RU} queries to $\mathcal{O}[\text{Reg.RA}^H]$, $\mathcal{O}[\text{Reg.RA}]$ and $\mathcal{O}[\text{Reg.RU}]$, respectively, and satisfies $\text{Adv}^{\text{RegComplete}}(\mathbf{A}) \geq \varepsilon$ (resp. $\text{Adv}^{\text{OW}}(\mathbf{A}) \geq \varepsilon$ and $\text{Adv}^{\text{WE}}(\mathbf{A}) \geq \varepsilon$). ROWR is

$$(\varepsilon_{\text{RegComplete}}, \varepsilon_{\text{OW}}, \varepsilon_{\text{WE}}, \gamma, t, n, q_{\text{RA}}^H, q_{\text{RA}}, q_{\text{RU}})\text{-secure}$$

if no adversary:

- $(\varepsilon_{\text{RegComplete}}, t)$ -breaks its $(n, q_{\text{RA}}^H, q_{\text{RA}}, q_{\text{RU}})$ -Registration-Completeness,
- $(\varepsilon_{\text{OW}}, t)$ -breaks its $(n, q_{\text{RA}}^H, q_{\text{RA}}, q_{\text{RU}})$ -One-Wayness nor
- $(\varepsilon_{\text{WE}}, t)$ -breaks its $(n, q_{\text{RA}}^H, q_{\text{RA}}, q_{\text{RU}})$ -Witness-Extractability,

and ROWR is γ -Statement-Spread.

5.3 ROWR with Simulatable Registration

A Registrable One-Way Relation with Simulatable Registration (ROWR-SR) for binary relation $R \subseteq \mathcal{X} \times \mathcal{W}$ is a tuple

$$\text{ROWR-SR} = (\text{Setup}, \text{Sim}_{\text{Setup}}, \text{Samp}, D_{\in R}, \text{Reg} := (\text{RU}, \text{RA}, \text{Sim}_{\text{RU}}, \text{Extract})),$$

where $(\text{Setup}, \text{Samp}, D_{\in R}, \text{Reg} := (\text{RU}, \text{RA}, \text{Extract}))$ is a Registrable One-Way Relation for R and:

- $\text{Sim}_{\text{Setup}}$: outputs public parameters pp and a trapdoor τ ;
- Sim_{RU} is an interactive algorithms such that

$$\langle b \mid x \rangle \leftarrow \langle \text{Sim}_{\text{RU}}(\text{pp}, \tau, x) \leftrightarrow \text{RA}(\text{pp}) \rangle,$$

where Sim_{RU} 's output is a bit b indicating if registration succeeded, and RA 's output is either x (RU 's input) or $\perp$ if registration fails.

These schemes are intended to provide all the guarantees of Registrable One-Way Relations. In addition, it is also expected that user registrations made via algorithms Reg.RU and $\text{Reg.Sim}_{\text{RU}}$ are indistinguishable. The (distinction-based) security notion that formalizes this extra guarantee relies on the oracles defined for Registrable One-Way Relation plus modified $\mathcal{O}[\text{Setup}^{\mathbf{b}}]$ and $\mathcal{O}[\text{Reg.RU}^{\mathbf{b}}]$ oracles (for $\mathbf{b} \in \{0, 1\}$). Oracles $\mathcal{O}[\text{Setup}^0]$ and $\mathcal{O}[\text{Reg.RU}^0]$ work just like the original ones for Registrable One-Way Relations; oracles $\mathcal{O}[\text{Setup}^1]$ and $\mathcal{O}[\text{Reg.RU}^1]$ are slightly different:

$\mathcal{O}[\text{Setup}^1]$: Sample $(\text{pp}, \tau) \leftarrow \text{Sim}_{\text{Setup}}$. Output pp .
 $\mathcal{O}[\text{Reg.RU}^1](i \in \mathbb{N}, \text{sid} \in \mathbb{N}, m)$: Just like $\mathcal{O}[\text{Reg.RU}^0]$, except that it creates $\text{Reg.Sim}_{\text{RU}}$ instances instead of Reg.RU instances. (These instances are given the trapdoor τ as input, but not the user's witnesses.)

Definition 6 (Simulatable Registration).

$$\vec{\mathcal{O}}_{\text{SimReg}}^{\mathbf{b}} := \vec{\mathcal{O}}[\text{Setup}^{\mathbf{b}}, \text{Samp}, \text{Reg.RA}^H, \text{Reg.RA}, \text{Reg.RU}^{\mathbf{b}}]$$

5.4 Instantiating Registrable One-Way Relations with Simulatable Registration

We can easily instantiate Registrable One-Way Relation with Simulatable Registration with any NP relation with a straightline simulatable zero-knowledge proof of knowledge where extraction happens without using a trapdoor. One example would be the discrete logarithm relation in finite group $\mathbb{G}$ of prime order q with generator g .

Let the setup algorithm sample a $b \leftarrow \mathbb{Z}_q$ and outputs $\text{pp} = g^b$. The simulated setup does the same but outputs $(\text{pp} = g^b, \tau = b)$. The sampling algorithm samples $a \leftarrow \mathbb{Z}_q$ and outputs $(x = g^a, w = a)$.

To register a statement x , the user can use the OR-composition [19] of Schnorr's identification protocol [41] to prove that they know a witness w , such that either $x = g^w$ or $\text{pp} = g^w$. Due to the special soundness this protocol is known to be extractable without a trapdoor. Further, since the OR-composition of Σ -protocols is witness-indistinguishable [19] the simulator can simulate by using the alternative witness from the public parameters in a straightline manner.

6 Registration-based Ring Signatures

We now study Ring Signatures and in particular define (what seem to be) the minimum requirements needed to enable plausible deniability guarantees for these schemes. A Ring Signature is a tuple of probabilistic algorithms $\text{RSig} := (\text{Setup}, \text{Gen}, \text{Reg} := (\text{RU}, \text{RA}, \text{Extract}), \text{Sign}, \text{Vrfy})$:

- **Setup**: outputs public parameters pp ;
- **Gen**(pp): outputs a key-pair (vk, sk) ;
- **Reg** $:= (\text{RU}, \text{RA}, \text{Extract})$ is a triple which is analogous to the one of Registrable One-Way Relations:
 1. (RU, RA) being interacting algorithms

$$\langle b \mid \text{vk} \rangle \leftarrow \langle \text{RU}(\text{pp}, \text{vk}, \text{sk}) \leftrightarrow \text{RA}(\text{pp}) \rangle,$$

where RU 's output is a bit b indicating if registration succeeded, and RA 's output is either vk (RU 's input) or $\perp$ if registration fails.

2. **Extract**($\text{pp}, U, \text{vk}, \text{Transc}, \vec{v}, m$): given an interactive algorithm's description U , a public key vk , a transcript Transc , a ring $\vec{v}$ (i.e. vector of verification keys) that includes vk and a message m , outputs a signature σ or $\perp$ (if signature extraction fails).
- **Sign**($\text{pp}, \text{sk}, \vec{v} := (\text{vk}_1, \dots, \text{vk}_n), m$): outputs a signature σ ;
- **Vrfy**($\text{pp}, \vec{v}, m, \sigma$): outputs a bit indicating the validity of σ (with respect to ring $\vec{v}$ and message m); bit 1 means σ is a valid signature and bit 0 otherwise.

6.1 Security Notions

Consider a $\text{RSig} := (\text{Setup}, \text{Gen}, \text{Reg} := (\text{RU}, \text{RA}, \text{Extract}), \text{Sign}, \text{Vrfy})$. The security notions defined ahead rely on oracles $\vec{\mathcal{O}}[\text{Setup}, \text{SK}, \text{VK}, \text{Reg.RA}, \text{Reg.RU}, \text{Sign}, \text{Vrfy}]$

which are (mostly) analogous to the ones we defined for ROWR. For completeness, we now define them:

$\mathcal{O}[\text{Setup}]$: If pp is undefined, sample $\text{pp} \leftarrow \text{Setup}$; output pp .
 $\mathcal{O}[\text{SK}](i \in \mathbb{N})$: if $(\text{vk}_i, \text{sk}_i)$ is undefined, sample $(\text{vk}_i, \text{sk}_i) \leftarrow \text{Gen}(\text{pp})$; output $(\text{vk}_i, \text{sk}_i)$.
 $\mathcal{O}[\text{VK}](i)$ $(\text{vk}_i, \text{sk}_i) \leftarrow \mathcal{O}[\text{SK}](i)$; output vk_i .
 $\mathcal{O}[\text{Reg.RA}^H](i \in \mathbb{N})$: Analogous to oracle $\mathcal{O}[\text{Reg.RA}^H]$ of Registrable One-Way Relations.
 $\mathcal{O}[\text{Reg.RA}](i \in \mathbb{N}, U)$: If vk_i is defined output $\perp$. Else, given the description of (stateful) interactive algorithm U , run $\langle \text{out} \mid \text{vk} \rangle \leftarrow \langle U \leftrightarrow \text{RA}(\text{pp}) \rangle$, and let $\text{Transc}_{\langle U \leftrightarrow \text{RA} \rangle}$ be the respective interaction transcript. If Reg.RA halted and output $\text{vk} \neq \perp$, then define $\text{vk}_i := \text{vk}$. Output $(\text{out}, \text{vk}, \text{Transc}_{\langle U \leftrightarrow \text{RA} \rangle})$.
 $\mathcal{O}[\text{Reg.RU}](i \in \mathbb{N}, \text{sid} \in \mathbb{N}, m)$: Analogous to Registrable One-Way Relations' oracle $\mathcal{O}[\text{Reg.RU}^H]$.
 $\mathcal{O}[\text{Sign}](i, \vec{V}, m)$: If $i \notin \vec{V}$, or if vk_i was defined by a query $\mathcal{O}[\text{Reg.RA}](i, \cdot)$, output $\perp$. For each $j \in \vec{V}$ such that vk_j is undefined, query $\mathcal{O}[\text{VK}](j)$. Output $\text{Sign}_{\text{pp}}(\text{sk}_i, \vec{v}, m)$.
 $\mathcal{O}[\text{Vrfy}](\vec{V}, m, \sigma)$: Output $\text{Vrfy}_{\text{pp}}(\vec{v}, m, \sigma)$.

Definition 7 (Correctness: $\neg \text{Corr}$). Fix some $i \in \mathbb{N}$. The output of every query $\mathcal{O}[\text{Reg.RA}](i, \cdot)$ is of the form $(\cdot, \text{vk} = \perp, \cdot)$. A query $\mathcal{O}[\text{Sign}](i, \vec{V}, m)$ with $i \in \vec{V}$ outputs a signature σ . A later query $\mathcal{O}[\text{Vrfy}](\vec{V}, m, \sigma)$ outputs 0.

Definition 8 (Registration-Completeness: $\neg \text{RegComplete}$). Verification key vk_i is defined by a query $\mathcal{O}[\text{VK}](i)$ or by a query $\mathcal{O}[\text{SK}](i)$. A query $\mathcal{O}[\text{Reg.RA}^H](i)$ outputs $\perp$ or a pair $(b, \text{vk}) \neq (1, \text{vk}_i)$.

Definition 9 (Unforgeability: $\neg \text{EUF-CMA}$). A query $\mathcal{O}[\text{Vrfy}](\vec{V}^*, m^*, \sigma^*)$ outputs 1. For each $i \in \vec{V}^*$: 1. the output of every query $\mathcal{O}[\text{Reg.RA}](i, \cdot)$ is of the form $(\cdot, \text{vk} = \perp, \cdot)$, and 2. there is no query $\mathcal{O}[\text{SK}](i)$ prior to the $\mathcal{O}[\text{Vrfy}]$ query. There are no queries $\mathcal{O}[\text{Sign}](i^*, \vec{V}^*, m^*)$ prior to the $\mathcal{O}[\text{Vrfy}]$ query.

The anonymity notion relies on the oracles defined above plus modified $\mathcal{O}[\text{Sign}^b]$ oracles, which work as follows:

$\mathcal{O}[\text{Sign}^b](i, \vec{V}, m)$: If vk_i is undefined, or, if vk_j is undefined for some $j \in \vec{V}$, then output $\perp$. If $\mathbf{b} = 0$, then let $l^* := i$ and $\text{vk}^* := \text{vk}_i$. Else, if $\mathbf{b} = 1$, let $l^* := V_0$ and $\text{vk}^* := v_0$.
If vk^* was defined by a query to either $\mathcal{O}[\text{SK}]$ or $\mathcal{O}[\text{VK}]$, then let sk^* be the respective secret key; compute $\sigma \leftarrow \text{Sign}_{\text{pp}}(\text{sk}^*, \vec{v}, m)$ and then output σ . Else, consider the $\mathcal{O}[\text{Reg.RA}](l^*, U)$ query that output $(\text{out}, \text{vk}^* \neq \perp, \text{Transc}_{\langle U \leftrightarrow \text{RA} \rangle})$. Compute $\sigma \leftarrow \text{Extract}(\text{pp}, U, \text{vk}^*, \text{Transc}_{\langle U \leftrightarrow \text{RA} \rangle}, \vec{v}, m)$. Output σ .

Definition 10 (Anon). $\tilde{\mathcal{O}}_{\text{Anon}}^b := \tilde{\mathcal{O}}[\text{Setup}, \text{SK}, \text{VK}, \text{Reg.RA}, \text{Reg.RU}, \text{Sign}^b, \text{Vrfy}]$.

6.2 Construction

We now show how to construct a Ring Signature scheme providing the guarantees defined above. Our construction is defined in Algorithm 1 and its building blocks are a Registrable One-Way Relation $\text{ROWR} = (\text{Setup}, \text{Samp}, D_{\in R}, \text{Reg} := (\text{RU}, \text{RA}, \text{Extract}))$ for some relation $R \subseteq \mathcal{X} \times \mathcal{W}$, a Non-Interactive Zero Knowledge $\text{NIZK} = (\text{Setup}, \text{Prv}, \text{Vrfy}, \text{Sim} = (\text{Sim.Setup}, \text{Sim.Prv}))$, a Public Key Encryption scheme $\text{CPA} = (\text{Gen}, \text{Enc}, \text{Dec})$ and a Hash Function $\text{HF} = (\text{Setup}, \text{H})$.⁷ The security of our construction naturally relies on the security of its building blocks, plus on an extra requirement on ROWR 's relation $R \subseteq \mathcal{X} \times \mathcal{W}$. Let $L_R \subseteq \mathcal{X}$ be the natural language induced by R and consider a (computable) function $\text{WitLen} : L_R \rightarrow \mathbb{N}$.

Definition 11 (WitLen-Upper Bounded Witness Length Relation). *We say binary relation $R \subseteq \mathcal{X} \times \mathcal{W}$ has WitLen upper bounded witness length if $\forall x \in L_R, \forall w \in \mathcal{W} : (x, w) \in R \rightarrow |w| \leq \text{WitLen}(x)$.*

Remark 4 (Runtime of WitLen). As one may note, we did not make any requirements on the runtime of WitLen nor on its output. To be in line with the asymptotic literature one could require the runtime to be bounded by some polynomial on the length of the statement input to WitLen , and the output length to be poly-logarithmic on the input length. However, the resulting definition would be unnecessarily more involved and in addition we do not know what the meaning of such bounds would be in practice.

This requirement translates into an assumption of the ROWR scheme. Namely, that a successful registration of a statement $x \in \mathcal{X}$ implies that the witness $w \in \mathcal{W}$ — with $(x, w) \in R$ — that can be extracted from the registration algorithm U input to $\mathcal{O}[\text{Reg.RA}]$ has length bounded by the function from Definition 11. Our construction will use the NIZK scheme to prove membership in the language L_{RS} induced by the following relation:

$$R_{\text{RS}} := \left\{ ((\text{pk}_{\text{CPA}}, h, c, \vec{x}), (w, \text{pad}, r)) : \bigvee_{i \in |\vec{x}|} \text{ROWR}.D_{\in R}(x_i, w) = 1 \quad (6.1) \right. \\ \left. \wedge c = \text{CPA}.Enc(\text{pk}_{\text{CPA}}, (w, \text{pad}, h); r) \right\}.$$

Security Analysis

Theorem 1 (Informal). *If 1. ROWR is Registration-Complete, One-Way and Witness-Extractable for a WitLen -Upper Bounded Witness Length Relation R , 2. NIZK is Complete, Zero Knowledge and Simulation Sound, 3. CPA is Correct and IND-CPA secure, and 4. HF is Collision Resistant, then RSig is*

⁷ Looking ahead, we note that the statements for which we will use the NIZK are (essentially) independent from HF . This is crucial to guarantee that there are efficient instantiations.

Algorithm 1 Construction of a Ring Signature from a Non-Interactive Zero Knowledge $\text{NIZK} = (\text{Setup}, \text{Prv}, \text{Vrfy}, \text{Sim} = (\text{Sim.Setup}, \text{Sim.Prv}))$, a Public Key Encryption scheme $\text{CPA} = (\text{Gen}, \text{Enc}, \text{Dec})$, a Hash Function $\text{HF} = (\text{Setup}, \text{H})$, and a Registrable One-Way Relation $\text{ROWR} = (\text{Setup}, \text{Samp}, D_{\in R}, \text{Reg} := (\text{RU}, \text{RA}, \text{Extract}))$ for a WitLen-Upper Bounded Witness Length Relation $R \subseteq \mathcal{X} \times \mathcal{W}$. For simplicity, the public parameters are implicitly input to all algorithms given below.

<pre> Setup crs $\leftarrow$ NIZK.Setup pp_{HF} $\leftarrow$ HF.Setup pp_{ROWR} $\leftarrow$ ROWR.Setup (pk_{CPA}, ·) $\leftarrow$ CPA.Gen return pp := (crs, pp_{HF}, pp_{ROWR}, pk_{CPA}) Gen: (x, w) $\leftarrow$ ROWR.Samp return (vk := x, sk := (vk, w)) Reg.RU(vk, sk) := ROWR.Reg.RU(vk, sk) Reg.RA := ROWR.Reg.RA Reg.Extract(U, vk, Transc, $\vec{v}$, m) if vk $\notin \vec{v}$: return $\perp$ w $\leftarrow$ ROWR.Reg.Extract(U, vk, Transc) if w = $\perp$: return $\perp$ return $\sigma \leftarrow \text{Sign}(\text{sk}^* := (\text{vk}, w), \vec{v}, m)$ </pre>	<pre> Sign(sk, $\vec{v}$, m) n := $\vec{v}$ if sk.vk $\notin \vec{v}$: return $\perp$ h $\leftarrow$ HF.H($\vec{v}$, m) l $\leftarrow \max_{i \in [n]} \text{WitLen}(v_i)$ pad $\leftarrow 0^{l - \text{sk}.w }$ c $\leftarrow$ CPA.Enc(pk_{CPA}, (sk.w, pad, h); r) $\pi \leftarrow \text{NIZK.Prv}((\text{pk}_{\text{CPA}}, h, c, \vec{v}) \in L_{\text{RS}}, (\text{sk}.w, \text{pad}, r))$ return $\sigma := (c, \pi)$ Vrfy($\vec{v}$, m, $\sigma := (c, \pi)$) h $\leftarrow$ HF.H($\vec{v}$, m) return NIZK.Vrfy((pk_{CPA}, h, c, $\vec{v}$) $\in L_{\text{RS}}, \pi)$ </pre>
---	---

1. Correct (Theorem 4),
2. Registration-Complete (Theorem 5),
3. Unforgeable (Theorem 6), and
4. Anonymity (Theorem 7),

all under adaptive corruptions, and with tight security reductions.

The Public Key Encryption scheme underlying our construction need not be non-malleable. The reason the security of our construction does not rely on this property is that it is already relying on the underlying NIZK to be Simulation Sound, and the statements that are proven using the NIZK together with its Simulation Soundness (informally) imply an adversary cannot maul R_{Sig}'s signatures into ones for different rings or messages. In other words, the underlying NIZK's Simulation Soundness makes R_{Sig}'s signatures non-malleable.

6.3 Relations with Witnesses of Unbounded Length

We note that our construction is restricted to working with relations that allow bounding the witness length based solely on the statement. This is necessary, because in our construction the witness is encrypted using a CPA secure public key encryption scheme. Since CPA security only holds for messages of the same length, we need to be able to pad all possible witnesses to the same length.

Consider as an example the relation $\mathcal{R} = \{(x, w) \mid x = H(w)\}$ for some collision resistant hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^{512}$. A malicious party could

choose w to be arbitrarily large, say 4GB. Since the signer could not tell from the statement that the witness is so large it would likely not choose to pad their witness to such a ridiculous length, thus allowing the malicious party to later easily prove that they did not sign based on size of the signature alone.

Therefore it is clearly a restriction, but we consider it a reasonable one. In fact, since we reduce unforgeability to one-wayness of the ROWR, we in fact know that we must be able to somehow extract the witness from the signature. At least in the standard model this implies that the signature must be at least as large as the witness, necessitating the restriction. It is worth noting, that in concurrent work [24] the same condition was apparently missed. Fiedler and Langrehr construct an anonymous DVS, where similarly a proof of knowledge is computed for a statement that includes an OR with a potentially maliciously chosen statement. Even though their paper states that the construction works for an arbitrary relation, a similar condition on the witness length is clearly required. This is true both in the standard model from an information theoretic perspective, as well as in practice for any circuit based PoK, where the size of the circuit will depend on the witness length.

7 Transitive Registrable One-Way Relations

This section introduces Transitive Registrable One-Way Relations, a useful building block to help amortizing the high setup cost required for deniable authentication. Consider some alphabet $\mathcal{E}$. A Transitive Registrable One-Way Relation (TROWR) for a family of relations $\{R_e \subseteq \mathcal{X}_e \times \mathcal{W}_e\}_{e \in \mathcal{E}^*}$ is a tuple of algorithms

$$\text{TROWR} = (\text{Setup}, \text{Samp}, D_{\in R}, \text{Reg} := (\text{RU}, \text{RA}, \text{Extract}), \text{TrExtend} := (\text{X}, \text{W}))$$

where $(\text{Setup}, \text{Samp}, D_{\in R}, \text{Reg} := (\text{RU}, \text{RA}, \text{Extract}))$ is a Registrable One-Way Relation for relation $R_{\mathcal{E}^*} := \bigcup_{s \in \mathcal{E}^*} R_s$ — the support of **Samp** being a subset of relation R_e , i.e. $\text{Supp}(\text{Samp}) \subseteq R_e$ — and:

- $\text{X}_{\text{pp}}(e \in \mathcal{E}^*, x \in \mathcal{X}_e)$: outputs a statement $x_e \in \mathcal{X}_e$;
- $\text{W}_{\text{pp}}(e \in \mathcal{E}^*, (x, w) \in R_s)$: for $u := s \parallel e$, outputs a witness $w_u \in \mathcal{W}_u$.

Both TrExtend.X and TrExtend.W are deterministic algorithms.⁸

Remark 5 (Syntax of TrExtend.X). As one may note, algorithm **X** only takes as input statements from the “empty string” relation $\mathcal{X}_\epsilon$. A (perhaps) more natural definition would allow **X** to extend possibly already extended statements, i.e. **X** could instead take as input statements $x_s \in \mathcal{X}_s$ — for any $s \in \mathcal{E}^*$ — plus an

⁸ Requiring these algorithms to be deterministic comes with no loss of generality if Pseudo-Random Functions exist. This is because one can construct a scheme for which these algorithms are deterministic from one for which these are probabilistic: concretely, one can make include a PRF seed together with each witness and then use this seed *a la* GGM [26] to obtain one for the next level.

extension $e \in \mathcal{E}^*$ and then for $u := s \parallel e$, output a statement $x_u \in \mathcal{X}_u$. However, our syntactical definition is sufficient for our purposes as users must know the “original” statements of other users. To see why, consider the Ring Signature application, and suppose Alice wants to sign a message m to a ring of parties $R = \{\text{Alice}, \text{Bob}\}$. Before Alice actually signs, she must first verify that Bob’s statement/public key has been certified by a registration authority she trusts — which implies she trusts that Bob knows the corresponding witness/secret key. (If Alice does not perform this check, she is not guaranteed plausible deniability.) Finally, we do not see why our definition would imply this alternative definition.

It is assumed that if X (resp. W) is input an empty extension string, then it simply outputs the statement (resp. witness) that was input.

7.1 Security Notions

The notions below rely on oracles $\vec{\mathcal{O}}[\text{Setup}, \text{Samp}, \text{Reg.RA}^H, \text{Reg.RA}, \text{Reg.RU}]$ defined for Registrable One-Way Relation schemes plus oracles $\vec{\mathcal{O}}[W^H, W, \epsilon]$:

$\mathcal{O}[W^H](i \in \mathbb{N}, \vec{e} \in (\mathcal{E}^*)^*)$:⁹ If x_i was defined by a query $\mathcal{O}[\text{Reg.RA}]$, output $\perp$. For $1 \leq j \leq |\vec{e}|$, let $u_j := e_1 \parallel \dots \parallel e_j$, $x^{u_j} \leftarrow \text{TrExtend.X}(\text{pp}, u_j, x_i)$ and compute $w_i^{u_j} \leftarrow \text{TrExtend.W}(\text{pp}, e_j, (x^{u_{j-1}}, w_i^{u_{j-1}}))$. Output 1 if $(x^{u_{|\vec{e}|}}, w_i^{u_{|\vec{e}|}}) \in R_{u_{|\vec{e}|}}$ and otherwise output 0.

$\mathcal{O}[W](i \in \mathbb{N}, e \in \mathcal{E}^*)$: If w_i is undefined output $\perp$. Output $\text{TrExtend.W}_{\text{pp}}(e, (x_i, w_i))$.

$\mathcal{O}[\epsilon](i, e, w_e)$: Output $(x_e := \text{TrExtend.X}(\text{pp}, e, x_i), w_e) \in R_e$.

Definition 12 (Transitive-One-Wayness: $\neg\text{TOW}$). Query $\mathcal{O}[\epsilon](i, e, w)$ outputs 1. The output of every query $\mathcal{O}[\text{Reg.RA}](i, \cdot)$ is of the form $(\cdot, x = \perp, \cdot, \cdot)$. There were no prior queries $\mathcal{O}[W](i, p)$ such that p is a prefix of e .

Definition 13 (Transitive-Witness-Extractability: $\neg\text{TWE}$). Some query $\mathcal{O}[\text{Reg.RA}](i, U)$ outputs $(\cdot, x, w, \cdot) \neq \perp$ with $x \neq \perp$. A query $\mathcal{O}[W](i, e)$ outputs some w_e . A query $\mathcal{O}[\epsilon](i, e, w_e)$ outputs 0.

Remark 6. Correctness for R_ϵ follows from Transitive-Witness-Extractability: one can make $\mathcal{O}[\text{Reg.RA}]$ queries with input $U = \text{Reg.RU}(\text{pp}, x_i, w_i)$ for (x_i, w_i) generated by the game. However, Correctness for extended relations does not follow: this is because there is no guarantee that TrExtend.W works correctly when input extended statement-witness pairs.

Definition 14 (TrExtend.W-Associativity: $\neg\text{WitAssoc}$). A query $\mathcal{O}[W^H](i, \vec{e})$ outputs 0.

Definition 15 (Registration-Completeness: $\neg\text{RegComplete}$). Some query $\mathcal{O}[\text{Samp}](i)$ defines statement x_i . A query $\mathcal{O}[\text{Reg.RA}^H](i)$ outputs $\perp$ or a pair $(b, x) \neq (1, x_i)$.

⁹ Vector $\vec{e}$ is a string whose alphabet are all finite strings over $\mathcal{E}$. In other words, each entry e_i of vector $\vec{e}$ is itself a string, $e_i \in \mathcal{E}^*$.

Definition 16 (γ -Statement-Spreadness). We say that a scheme $\text{TROWR} = (\text{Setup}, \text{Samp}, D_{\in R}, \text{Reg} := (\text{RU}, \text{RA}, \text{Extract}), \text{TrExtend} := (\text{X}, \text{W}))$ for relation family $\{R_e \subseteq \mathcal{X}_e \times \mathcal{W}_e\}_{e \in \mathcal{E}^*}$ with extension alphabet $\mathcal{E}$ is γ -Statement-Spread if $\forall \text{pp} \in \text{Supp}(\text{Setup}) : -\log \max_{x \in \mathcal{X}_e} \Pr_{r_\S}[x = \text{Samp}(\text{pp}; r_\S)] \geq \gamma$.

7.2 Construction

We show how to construct a Transitive Registrable One-Way Relation from Append-Only Signatures [31] (defined ahead) plus a Registrable One-Way Relation with Simulatable Registration. The relation for the ROWR-SR consists of verification keys of the Append-Only Signature, and witnesses as sequences of random coins. A verification-key randomness pair is in the ROWR-SR's relation if running Append-Only Signature's setup algorithm Setup with the given randomness results in a pair $(\text{vk}, \sigma_\varepsilon)$ such that vk matches the verification key in the ROWR-SR's statement.

Append-Only Signatures [31] Following [31], an Append-Only Signature is a tuple $\text{AOS} = (\text{Setup}, \text{Append}, \text{Vrfy})$ where Setup and Append are probabilistic algorithms and Vrfy is a deterministic algorithm. We denote the verification key space of an Append-Only Signature scheme by $\mathcal{V}$, the (message) alphabet space by $\mathcal{M}$ and the signature space by $\mathcal{S}$.

Setup : outputs a verification key vk and a signature on the empty string σ_ε ;
 $\text{Append}(\text{vk}, \sigma_s, m \in \mathcal{M})$: outputs a signature $\sigma_{s||m}$;
 $\text{Vrfy}(\text{vk}, \vec{m}, \sigma)$: outputs whether σ is valid with respect to vk and message $\vec{m}$.

For simplicity, we will abuse notation and write $\text{Append}(\text{vk}, \sigma_s, \vec{m} \in \mathcal{M}^*)$ to denote the iterative application of Append to each character of $\vec{m}$.

Remark 7 (Making AOS.Append deterministic). It is important to note that one can easily transform an AOS scheme with probabilistic Append algorithm into one with a deterministic Append algorithm — provided there is a Pseudo-Random Function with domain that matches AOS's message space and a co-domain consisting of bitstrings that are long enough so one can extract enough “independent” random coins to run both Append and to obtain a seed for the next level. This is important to note because our Transitive Registrable One-Way Relation construction uses as building block an Append-Only Signature, and requires that its Append algorithm is deterministic.

Definition 17 (Perfect Correctness). $\forall (\text{vk}, \sigma_\varepsilon) \in \text{Supp}(\text{Setup}), \forall \vec{m} \in \mathcal{M}^* : \text{Supp}(\text{Vrfy}(\text{vk}, \vec{m}, \text{Append}(\text{vk}, \vec{m}, \sigma_\varepsilon))) = \{1\}$.

The Unforgeability notion provides adversaries with the following oracles:

$\mathcal{O}[\text{Setup}]$: If vk is undefined, define $(\text{vk}, \sigma^\varepsilon) \xleftarrow{\S} \text{Gen}$. Output vk .

$\mathcal{O}[\text{Sign}](\vec{m} \in \mathcal{M}^*)$: If $\sigma^{\vec{m}}$ was already defined by a prior query, output $\sigma^{\vec{m}}$. Else, let $\vec{p}$ be the longest prefix of $\vec{m}$ for which signature $\sigma^{\vec{p}}$ was already defined. For $|\vec{p}| < j \leq |\vec{m}|$, compute and define $\sigma^{\vec{m}[1..j]} \leftarrow \text{Append}(\text{vk}, \sigma^{\vec{m}[1..j-1]}, m_j)$. Output $\sigma^{\vec{m}}$.
 $\mathcal{O}[\text{Vrfy}](\vec{m}, \sigma)$: Output $\text{Vrfy}_{\text{pp}}(\text{vk}, \vec{m}, \sigma)$.

Definition 18 (Unforgeability [31]: $\neg\text{Prefix-EUF-CMA}$). A query $\mathcal{O}[\text{Vrfy}](\vec{m}, \sigma)$ outputs 1. There is no prior query $\mathcal{O}[\text{Sign}](m[1..n])$, i.e. on a prefix of $\vec{m}$.

The following AOS security property is new, and in particular it was not captured by the original security model from [31].

Definition 19 (SigBound-Upper Bounded Signature Size). A scheme $\text{AOS} = (\text{Setup}, \text{Append}, \text{Vrfy})$ with message alphabet $\mathcal{M}$ has SigBound upper bounded signature sizes if $\forall(\text{vk}, \sigma_\varepsilon), \forall \vec{m} \in \mathcal{M}^*, \forall \sigma_{\vec{m}} \in \text{Supp}(\text{Append}(\text{vk}, \vec{m}, \sigma_\varepsilon))$: $|\sigma_{\vec{m}}| \leq \text{SigBound}(\text{vk}, \vec{m})$.

We note that natural instantiations of the Append-Only Signature constructions given in [31] enjoy this property.

Remark 8. We note that our TROWR construction crucially relies on the perfect correctness of the underlying Append-Only Signature. If this were not the case, an attacker could potentially pick a very specific pair $(\text{vk}, \sigma_\varepsilon)$ so that it cannot use AOS's Append algorithm to extend the signature on the empty string. If this were the case, an attacker could potentially break plausible deniability for messages that are authenticated with respect to extensions of its public key. This is because it could happen that he had the ability to later show that his key-pair is malformed.

Algorithm 2 The TROWR construction below is defined for the family of relations defined in Equation 7.1 and Equation 7.2. As building blocks, it relies on an Append-Only Signature $\text{AOS} = (\text{Setup}, \text{Append}, \text{Vrfy})$ and a Registrable One-Way Relation with Simulatable Registration $\text{ROWR-SR} = (\text{Setup}, \text{Sim}_{\text{Setup}}, \text{Samp}, D_{\in R}, \text{Reg} := (\text{RU}, \text{RA}, \text{Sim}_{\text{RU}}, \text{Extract}))$. For simplicity, below the public parameters are implicit inputs to all algorithms.

Setup := ROWR-SR.Setup	$D_{\in R}((\text{vk}, \vec{m}), w_{\vec{m}})$ if $\vec{m} = \varepsilon$: return ROWR-SR. $D_{\in R}(\text{vk}, w_{\vec{m}})$ else: return AOS.Vrfy(vk, $\vec{m}$, $w_{\vec{m}}$)
Reg.RU := ROWR-SR.Reg.RU	
Reg.RA := ROWR-SR.Reg.RA	
Reg.Extract := ROWR-SR.Reg.Extract	
Samp(r_s): $(\text{vk}, \cdot) \leftarrow \text{AOS.Setup}(r_s)$ return $(x_\varepsilon := (\text{vk}, \varepsilon), w_\varepsilon := r_s)$	TrExtend.X($\vec{m}$, (vk, ε)): return (vk, $\vec{m}$) TrExtend.W($\vec{m}$, ($x := (\text{vk}, s), w$)) if $s = \varepsilon$: $(\text{vk}, \sigma_s) \leftarrow \text{AOS.Setup}(w)$ else: $\sigma_s := w$ return AOS.Append(vk, σ_s , $\vec{m}$)

Construction from Append-Only Signatures We now show how to construct a Transitive Registrable One-Way Relation for a relation family $\{R_s^{\text{AOS}}\}_{s \in \mathcal{M}^*}$

from an Append-Only Signature scheme with message space $\mathcal{M}$ and a Registrable One-Way Relation with Simulatable Registration for relation $R_\varepsilon^{\text{AOS}}$, which is a member of the TROWR relation family and is formally defined as:

$$R_\varepsilon^{\text{AOS}} := \{((\mathbf{vk}, \varepsilon), r_\S) \mid (\mathbf{vk}, \sigma_\varepsilon) = \text{AOS.Setup}(r_\S)\}. \quad (7.1)$$

The remainder of the relation elements of our construction's family are as follows:

$$R_{s \in \mathcal{M}^+}^{\text{AOS}} := \{((\mathbf{vk}, s), \sigma_s) \mid \text{AOS.Vrfy}(\mathbf{vk}, s, \sigma_s) = 1\} \quad (7.2)$$

The construction is formally defined in Algorithm 2.

Security Analysis Below we give the main idea for the argument establishing the security of our TROWR construction. (See Section C for the proof.)

Theorem 2. *If*

- AOS is Perfectly Correct and Unforgeable, and
- ROWR-SR is Registration-Complete, One-Way, Witness-Extractable and Registration Simulatable for a relation R ,

then TROWR is Registration-Complete, Transitive-One-Way, Transitive-Witness-Extractable and TrExtend.W-Associativity. The reduction from breaking Transitive-One-Wayness of TROWR to breaking the Unforgeability of AOS has a security loss that is linear in the number of TROWR statements that are defined via oracle queries to $\mathcal{O}[\text{Samp}]$.

Proof (Proof Idea). The proof of this theorem is mostly straightforward from the definitions. The main things to note are:

- our construction's TrExtend.W algorithm is only deterministic if algorithm Append of the underlying AOS is deterministic. Although at first this may seem like a mismatch to the definition of an Append-Only Signature, we note that, as explained in Remark 7 one can transform any AOS scheme with a probabilistic Append algorithm into one with a deterministic Append algorithm.
- the Transitive-One-Wayness of our construction crucially relies on the Simulatable Registration security of the underlying ROWR-SR: this is necessary because otherwise we do not know how to handle queries to $\mathcal{O}[\text{Reg.RU}]$, since these require knowledge of the random coins used to run AOS' Setup algorithm. Then, once $\mathcal{O}[\text{Reg.RU}]$ can be handled without knowledge of ROWR-SR's witnesses, we can reduce to the Unforgeability of the underlying AOS. $\square$

As for our Ring Signature scheme, the security of the Transitive Ring Signature scheme we construct relies on an extra assumption on the TROWR's relation family. Namely it requires that TROWR's relation family is a ExtWitLen-Upper Bounded Witness Length Relation Family, as defined below. Recall that the natural language induced by a relation R is denoted L_R .

Definition 20 (ExtWitLen-Upper Bounded Witness Length Relation).
Family $\{R_e \subseteq \mathcal{X}_e \times \mathcal{W}_e\}_{e \in \mathcal{E}^}$ has ExtWitLen upper bounded witness length if*

$$\forall e \in \mathcal{E}^*, \forall x_e \in L_{R_e}, \forall w_e \in \mathcal{W}_e : (x_e, w_e) \in R_e \rightarrow |w_e| \leq \text{WitLen}(x_e, e).$$

It is easy to see that our TROWR construction does satisfy this property, provided the AOS scheme has SigBound-Upper Bounded Signature Sizes, and provided that one restricts the randomness space used by AOS’s Setup algorithm. (This is not a real restriction because one can always rely on a PRG to expand short seeds into enough randomness for AOS’s Setup algorithm.)

Instantiation To instantiate the above construction, we need to choose instantiations for the AOS as well as the associated ROWR-SR. One simple instantiation can be based on the certificate based AOS construction of Kiltz et al. [31]. This construction is based on essentially constructing a certificate chain and can be instantiated from any regular digital signature scheme. If we choose a digital signature scheme where public keys are group elements and the random coins of the key generation correspond to the discrete log of those group elements, we can then use the rowr instantiation described in Section 5.4 to instantiate the ROWR-SR. Depending on whether we are willing to work in the random oracle model or require a standard model instantiation, we can thus instantiate the AOS e.g. from BLS signatures [16] or Boneh-Boyen signatures [15]. (For BLS signatures, the NIZK statements that we would need to prove for our Ring Signature construction would not have to deal with the computation of the hash-function. This is because the extensions of each user’s public key are public, so one could “hard-code” the resulting group elements into the NIZK statements — which would be recomputed during verification.)

8 Knowledge-Transitive Ring Signatures

Consider some alphabet $\mathcal{E}$. A Transitive Ring Signature (TrRSig) for a family of relations $\{R_e \subseteq \mathcal{V}_e \times \mathcal{S}_e\}_{e \in \mathcal{E}^*}$ is a tuple of probabilistic algorithms

$$\text{TrRSig} := (\text{Setup}, \text{Gen}, \text{Reg} := (\text{RU}, \text{RA}, \text{Extract}), \text{Sign}, \text{Vrfy}, \text{TrExtend} := (\text{Vk}, \text{Sk}))$$

where $(\text{Setup}, \text{Gen}, \text{Reg} := (\text{RU}, \text{RA}, \text{Extract}), \text{Sign}, \text{Vrfy})$ is essentially a Ring Signature (a formal meaning of this is given ahead) and with

- $\text{Vk}_{\text{pp}}(e \in \mathcal{E}^*, \text{vk} \in \mathcal{V}_e)$: outputs an extended verification key $\text{vk}_e \in \mathcal{V}_e$;
- $\text{Sk}_{\text{pp}}(e \in \mathcal{E}^*, (\text{vk}, \text{sk}) \in R_s)$: for $u := s \parallel e$, outputs a secret key $\text{sk}_u \in \mathcal{S}_u$.

As for Transitive Registrable One-Way Relation, both TrExtend.Vk and TrExtend.Sk are deterministic algorithms. Above, we wrote “*essentially*” because the syntax of Sign , Vrfy and Reg.Extract is slightly different from the Ring Signature syntax we introduced before:

- $\text{Sign}(\text{pp}, \text{sk} \in \mathcal{S}_s, \vec{v} := (\text{vk}_1, \dots, \text{vk}_n), \vec{e} := (e_1, \dots, e_n), m)$: given a secret key from set $\mathcal{S}_s$, a public key vector $\vec{v}$ — where $v_i \in \mathcal{V}_e$ for each $i \in [n]$ — a message m and a vector of strings $\vec{e}$ — where $e_i \in \mathcal{E}^*$ for each $i \in [n]$ — outputs a signature σ . (Looking ahead, Correctness for Transitive Ring Signature will only apply if, informally, sk can be extended — possibly by the empty string ε — so that it matches some verification key in $\vec{v}$. A little more formally, Correctness only applies if for some $i \in [n]$, the secret key $\text{sk} \in \mathcal{S}_s$ that is input, or, an TrExtend.Sk extension of it, together with an TrExtend.Vk extension of v_i with respect to extension string e_i , is an element of R_{e_i} .)
- $\text{Vrfy}(\text{pp}, \vec{v}, \vec{e}, m, \sigma)$: outputs a bit indicating the validity of σ (with respect to ring $\vec{v}$ extended according to $\vec{e}$ and message m).
- $\text{Reg.Extract}(\text{pp}, U, \text{vk}, \text{Transc}, \vec{v}, \vec{e}, m)$: given an interactive algorithm's description U , a public key vk , a transcript Transc , a ring $\vec{v}$ and corresponding extension vector $\vec{e}$ (where $\vec{v}$ includes vk) and a message m , outputs a signature σ or $\perp$ (if signature extraction fails).

Concretely, we require that, for algorithms Sign_ε , Vrfy_ε and $\text{TrExtend.Extract}_\varepsilon$ defined as Sign , Vrfy and TrExtend.Extract with extension vector being fixed to a vector of empty strings, i.e. $\vec{e} := (\varepsilon, \dots, \varepsilon)$ and, for Sign , with the additional restriction that sk is an element of $\mathcal{S}_\varepsilon$, the tuple

$$(\text{Setup}, \text{Gen}, \text{Reg} := (\text{RU}, \text{RA}, \text{Extract}_\varepsilon), \text{Sign}_\varepsilon, \text{Vrfy}_\varepsilon)$$

is a Ring Signature with $\text{Supp}(\text{Gen}(\text{Setup})) \subseteq R_\varepsilon$ (i.e. the support of Gen being a subset of R_ε). As for Transitive Registrable One-Way Relations, it is assumed that if Vk (resp. Sk) is input an empty extension string, then it outputs the verification key (resp. signing key) that was input.

8.1 Security Notions

Consider some alphabet $\mathcal{E}$ and a Transitive Ring Signature

$$\text{TrRSig} := (\text{Setup}, \text{Gen}, \text{Reg} := (\text{RU}, \text{RA}, \text{Extract}), \text{Sign}, \text{Vrfy}, \text{TrExtend} := (\text{Vk}, \text{Sk}))$$

for a family of relations $\{R_e \subseteq \mathcal{V}_e \times \mathcal{S}_e\}_{e \in \mathcal{E}^*}$. The notions ahead rely on oracles $\vec{\mathcal{O}}[\text{Setup}, \text{VK}, \text{Reg.RA}^H, \text{Reg.RA}, \text{Reg.RU}]$ defined for Ring Signatures, plus to oracles $\vec{\mathcal{O}}[\text{SK}, \text{Sign}, \text{Vrfy}]$:

- $\mathcal{O}[\text{SK}](i \in \mathbb{N}, s \in \mathcal{E}^*)$: if vk_i was defined by a query to Reg.RA , output $\perp$. If $(\text{vk}_i, \text{sk}_i)$ is undefined, sample $(\text{vk}_i, \text{sk}_i) \leftarrow \text{Gen}(\text{pp})$. Compute $\text{sk}_{i,s} \leftarrow \text{Sk}_{\text{pp}}(e, (\text{vk}_i, \text{sk}_i))$; output $\text{sk}_{i,s}$.
- $\mathcal{O}[\text{Sign}](i, s \in \mathcal{E}^*, \vec{V}, \vec{e}, m)$: If $i \notin \vec{V}$ or vk_i was defined by a query $\mathcal{O}[\text{Reg.RA}](i, \cdot)$, output $\perp$. For each $j \in \vec{V}$ such that vk_j is undefined, query $\mathcal{O}[\text{VK}](j)$. Compute $\text{sk}_{i,s} \leftarrow \text{TrExtend.Sk}_{\text{pp}}(s, (\text{vk}_i, \text{sk}_i))$. Output $\text{Sign}_{\text{pp}}(\text{sk}_{i,s}, \vec{v}, \vec{e}, m)$.
- $\mathcal{O}[\text{Vrfy}](\vec{V}, \vec{e}, m, \sigma)$: Output $\text{Vrfy}_{\text{pp}}(\vec{v}, \vec{e}, m, \sigma)$.

Definition 21 (Correctness: $\neg\text{Corr}$). Fix some $i \in \mathbb{N}$. The output of every query $\mathcal{O}[\text{Reg.RA}](i, \cdot)$ is of the form $(\cdot, \text{vk} = \perp, \cdot)$. A query $\mathcal{O}[\text{Sign}](i, s, \vec{V}, \vec{e}, m)$ with $i \in \vec{V}$ outputs a signature σ . A later query $\mathcal{O}[\text{Vrfy}](\vec{V}, \vec{e}, m, \sigma)$ outputs 0.

Definition 22 (Registration-Completeness: $\neg\text{RegComplete}$). Verification key vk_i is defined by a query $\mathcal{O}[\text{VK}](i)$ or by a query $\mathcal{O}[\text{SK}](i, \cdot)$. A query $\mathcal{O}[\text{Reg.RA}^H](i)$ outputs $\perp$ or a pair $(b, \text{vk}) \neq (1, \text{vk}_i)$.

Definition 23 (Unforgeability: $\neg\text{EUF-CMA}$). A query $\mathcal{O}[\text{Vrfy}](\vec{V}^*, \vec{e}^*, m^*, \sigma^*)$ outputs 1. For each $i \in \vec{V}^*$: 1. the output of every query $\mathcal{O}[\text{Reg.RA}](i, \cdot)$ is of the form $(\cdot, \text{vk} = \perp, \cdot)$, and 2. there is no query $\mathcal{O}[\text{SK}](i, s)$ prior to the $\mathcal{O}[\text{Vrfy}]$ query such that s is a prefix of e_i^* . There are no queries $\mathcal{O}[\text{Sign}](i^*, \cdot, \vec{V}^*, \vec{e}^*, m^*)$ prior to the $\mathcal{O}[\text{Vrfy}]$ query.

The anonymity notion relies on the oracles defined above plus modified $\mathcal{O}[\text{Sign}^b]$ oracles, which work as follows:

$\mathcal{O}[\text{Sign}^b](i, s, \vec{V}, \vec{e}, m)$: If vk_i is undefined, or, if vk_j is undefined for some $j \in \vec{V}$, then output $\perp$. If $b = 0$, then let $l^* := i$ and $\text{vk}^* := \text{vk}_i$. Else, if $b = 1$, let $l^* := V_0$ and $\text{vk}^* := v_0$. If vk^* was defined by a query to either $\mathcal{O}[\text{SK}]$ or $\mathcal{O}[\text{VK}]$, then let sk^* be the respective secret key; compute $\text{sk}_s^* \leftarrow \text{TrExtend.Sk}_{\text{pp}}(s, (\text{vk}^*, \text{sk}^*))$; then compute $\sigma \leftarrow \text{Sign}_{\text{pp}}(\text{sk}_s^*, \vec{v}, \vec{e}, m)$. Else, consider the $\mathcal{O}[\text{Reg.RA}](l^*, U)$ query that output $(\text{out}, \text{vk}^* \neq \perp, \text{Transc}_{\langle U \leftrightarrow \text{RA} \rangle})$. Compute a signature $\sigma \leftarrow \text{Reg.Extract}(\text{pp}, U, \text{vk}^*, \text{Transc}_{\langle U \leftrightarrow \text{RA} \rangle}, \vec{v}, \vec{e}, m)$. In both cases output σ .

Definition 24 (Anon). $\vec{\mathcal{O}}_{\text{Anon}}^b := \vec{\mathcal{O}}[\text{Setup}, \text{SK}, \text{VK}, \text{Reg.RA}, \text{Reg.RU}, \text{Sign}^b, \text{Vrfy}]$.

8.2 Construction

Finally, we give a construction of a Transitive Ring Signature, which is very similar to our Ring Signature scheme, and is formally defined in Algorithm 3.

Consider an extension alphabet $\mathcal{E}$. The building blocks of our construction are a Transitive Registrable One-Way Relation $\text{TROWR} = (\text{Setup}, \text{Samp}, D_{\in R}, \text{Reg} := (\text{RU}, \text{RA}, \text{Extract}), \text{TrExtend} := (\text{X}, \text{W}))$ for relation family $\{R_e \subseteq \mathcal{X}_e \times \mathcal{W}_e\}_{e \in \mathcal{E}^*}$, a NIZK $= (\text{Setup}, \text{Prv}, \text{Vrfy}, \text{Sim} = (\text{Sim.Setup}, \text{Sim.Prv}))$, a PKE scheme $\text{CPA} = (\text{Gen}, \text{enc}, \text{Dec})$ and a Hash Function $\text{HF} = (\text{Setup}, \text{H})$. Analogously to our Ring Signature scheme, we require that TROWR's relation family is a ExtWitLen-Upper Bounded Witness Length Relation Family (see Definition 20).

Our construction's extension alphabet is the same as the underlying TROWR's extension alphabet; its relation family is also defined based on the underlying TROWR's relation family. To be concrete, suppose TROWR is a Transitive Registrable One-Way Relation for relation family $\{R_e^{\text{TROWR}} \subseteq \mathcal{X}_e^{\text{TROWR}} \times \mathcal{W}_e^{\text{TROWR}}\}_{e \in \mathcal{E}^*}$. For each extension string $e \in \mathcal{E}^*$, relation $R_e \subseteq \mathcal{V}_e \times \mathcal{S}_e$ of our construction's relation family is defined as follows:

$$\begin{aligned} \mathcal{V}_e &:= \mathcal{X}_e^{\text{TROWR}} & \mathcal{S}_e &:= \mathcal{X}_e^{\text{TROWR}} \times \mathcal{W}_e^{\text{TROWR}} \times \mathcal{E}^* \\ R_e &:= \{(x', (x, w, s)) \mid x' = x \wedge s = e \wedge (x, w) \in R_e^{\text{TROWR}}\} \end{aligned} \quad (8.1)$$

Our construction will use the NIZK to prove membership in language L_{TRS} which is defined very similarly to our Ring Signature scheme's language L_{RS} . For completeness, we define relation R_{TRS} — corresponding to L_{TRS} — below:

$$R_{\text{TRS}} := \left\{ ((\text{pk}_{\text{CPA}}, h, c, \vec{x}), (w, \text{pad}, r)) : \bigvee_{i \in |\vec{x}|} \text{TROWR}.D_{\in R}(x_i, w) = 1 \right. \\ \left. \wedge c = \text{CPA}.Enc(\text{pk}_{\text{CPA}}, (w, \text{pad}, h); r) \right\}. \quad (8.2)$$

Algorithm 3 Construction of TrRSig for the relation family defined in Equation 8.1. The structure and main ideas behind this scheme are the same as the ones behind our Ring Signature construction, and it also uses the same building blocks as RSig Algorithm 1, except that the ROWR scheme is replaced by a Transitive Registrable One-Way Relation TROWR = (Setup, Samp, $D_{\in R}$, Reg := (RU, RA, Extract), TrExtend := (X, W)) for a ExtWitLen-Upper Bounded Witness Length Relation Family $\{R_e \subseteq \mathcal{X}_e \times \mathcal{W}_e\}_{e \in \mathcal{E}^*}$ where $\mathcal{E}$ is the extension alphabet.

<pre> Setup crs $\leftarrow$ NIZK.Setup pp_{HF} $\leftarrow$ HF.Setup pp_{TROWR} $\leftarrow$ TROWR.Setup (pk_{CPA}, ·) $\leftarrow$ CPA.Gen return pp := (crs, pp_{HF}, pp_{TROWR}, pk_{CPA}) Gen: (x, w) $\leftarrow$ TROWR.Samp return (vk := x, sk := (x, w, ε)) Reg.RU(vk, sk) := TROWR.Reg.RU(vk, sk.w) Reg.RA := TROWR.Reg.RA Reg.Extract(U, vk, Transc, $\vec{v}$, $\vec{e}$, m) if vk $\notin \vec{v}$: return $\perp$ w_ε $\leftarrow$ TROWR.Reg.Extract(U, vk, Transc) if w_ε = $\perp$: return $\perp$ return Sign(sk* := (vk, w_ε, ε), $\vec{v}$, $\vec{e}$, m) TrExtend.Vk(e, vk) return TROWR.TrExtend.X(e, vk) TrExtend.Sk(e, (vk_s, sk := (vk_s, w_s, s))) w_{s e} $\leftarrow$ TROWR.TrExtend.W(e, (vk_s, w_s)) return (vk_s, w_{s e}, s e) </pre>	<pre> Sign(sk := (vk_s, w, s), $\vec{v}$, $\vec{e}$, m) n := $\vec{v}$ if $\nexists j \in [n] : (\text{vk}_s = v_j) \wedge \text{PrefixOf}(s, e_j)$: return $\perp$ x_s $\leftarrow$ TROWR.TrExtend.X(s, (vk_s, s)) Let j be the least index satisfying the condition above Let s_j $\in \mathcal{E}^*$ be such that: e_j = s s_j w_{e_j} $\leftarrow$ TROWR.TrExtend.W(s_j, (x_s, w_s)) for i $\in [n]$: x_i $\leftarrow$ TROWR.TrExtend.X(e_i, v_i) $\vec{x}$:= (x₁, ..., x_n) l $\leftarrow \max_{i \in [n]} \text{ExtWitLen}(x_i, e_i)$ pad $\leftarrow 0^{l - w_{e_j} }$ h $\leftarrow$ HF.H($\vec{v}$, $\vec{e}$, m) c $\leftarrow$ CPA.Enc(pk_{CPA}, (w_{e_j}, pad, h); r) π $\leftarrow$ NIZK.Priv((pk_{CPA}, h, c, $\vec{x}$) $\in$ L_{TRS}, (w_{e_j}, pad, r)) return σ := (c, π) Vrfy($\vec{v}$, $\vec{e}$, m, σ := (c, π)) for i $\in [n]$: x_i $\leftarrow$ TROWR.TrExtend.X(e_i, v_i) $\vec{x}$:= (x₁, ..., x_n) h $\leftarrow$ HF.H($\vec{v}$, $\vec{e}$, m) return NIZK.Vrfy((pk_{CPA}, h, c, $\vec{x}$) $\in$ L_{TRS}, π) </pre>
--	---

Security Analysis

Theorem 3 (Informal). *If 1. TROWR is Registration-Complete, TrExtend.W-Associativity, Transitive-One-Wayness and Transitive-Witness-Extractability for a ExtWitLen-Upper Bounded Witness Length Relation family $\{R_e \subseteq \mathcal{X}_e \times \mathcal{W}_e\}_{e \in \mathcal{E}^*}$, 2. NIZK is Complete, Zero Knowledge and Simulation Sound, 3. CPA is Correct*

and IND-CPA secure, and 4. HF is Collision Resistant, then TrRSig is Registration-Complete, Correct, Unforgeable and Anonymity, all under adaptive corruptions, and with tight security reductions.

Proof. The same arguments used to prove the security of our RSig construction also apply to our TrRSig construction, with minor modifications. Below, we give an overview of these main differences:

Correctness proof. For RSig we relied on ROWR’s sampler outputting statement-witness pairs in the ROWR’s relation, and then argued that Correctness follows from the underlying’s NIZK’s Completeness. For TROWR, however, we additionally need to rely on the underlying TROWR’s TrExtend.W-Associativity since we need to guarantee correctness even when an “extended” secret key is input to generate a signature.

Unforgeability proof. The entire sequence of hybrids (and each reduction between these hybrids) still applies. The main difference is the last reduction, which in this case is to the Transitive-One-Wayness of TROWR. However, even this reduction is still rather straightforward. One only needs to argue that a signature forgery that triggers event $\neg\text{EUF-CMA}$ implies that a straightforward reduction to TROWR’s Transitive-One-Wayness does trigger TROWR’s $\neg\text{TOW}$ event.

Anonymity proof. The only difference from our Ring Signature construction’s proof is that the signature extraction algorithm now needs to extend the witness w_ε that it obtains from TROWR’s Reg.Extract algorithm.

The argument for Registration-Completeness is the same as for our Ring Signature construction. $\square$

Instantiation To instantiate the above construction, we need to choose instantiations for the TROWR as well as the encryption scheme, the hash function, and the NIZK. The public key encryption scheme can be instantiated with a hybrid encryption scheme based on the Diffie-Hellman KEM [22] and AES [5]. The hash function can be instantiated using any standard hash function such as SHA-256 [42] or SHA-3 [43]. The simulation sound NIZK can then be instantiated e.g. with a simulation sound SNARK [10] in the ROM to achieve constant size signatures or using [27] which leads to an instantiation in the standard model albeit with significantly larger signatures.

The main focus of this work is not on providing actually practical solutions. The instantiations we discuss are intended to hint that one might be able to obtain practical instantiations of the schemes we introduce — though this will require significant optimizations.

Acknowledgments

The authors would like to thank Eike Kiltz and Vadim Lyubashevsky for helpful discussions. Guilherme Rito was supported by the European Union (ERC AdG REWORC - 101054911).

References

1. Certification practice statement (cps).
https://www.microsoft.com/pkiops/Docs/Content/policy/Microsoft_PKI-Services_CPS_v3.2.1.pdf (jan 2022), [Accessed 14-10-2025]
2. Baseline requirements for the issuance and management of publicly-trusted code signing certificates.
https://www.microsoft.com/pkiops/Docs/Content/policy/Microsoft_PKI-Services_CPS_v3.2.1.pdf (aug 2024), [Accessed 14-10-2025]
3. Certification practice statement.
https://images.apple.com/certificateauthority/pdf/Apple_Public_CA-CPS_v6.2.pdf (mar 2024), [Accessed 14-10-2025]
4. Registration authority practices statement (rps).
<https://www.digicert.com/content/dam/digicert/pdfs/legal/digicert-private-pki-rps-v1.3.pdf> (jul 2024), [Accessed 14-10-2025]
5. Advanced Encryption Standard (AES). National Institute of Standards and Technology, NIST FIPS PUB 197, U.S. Department of Commerce (Nov 2001)
6. Alwen, J., Coretti, S., Dodis, Y., Tselekounis, Y.: Security analysis and improvements for the IETF MLS standard for group messaging. In: Micciancio, D., Ristenpart, T. (eds.) *Advances in Cryptology – CRYPTO 2020, Part I. Lecture Notes in Computer Science*, vol. 12170, pp. 248–277. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 17–21, 2020). https://doi.org/10.1007/978-3-030-56784-2_9
7. Alwen, J., Coretti, S., Jost, D., Mularczyk, M.: Continuous group key agreement with active security. In: Pass, R., Pietrzak, K. (eds.) *TCC 2020: 18th Theory of Cryptography Conference, Part II. Lecture Notes in Computer Science*, vol. 12551, pp. 261–290. Springer, Cham, Switzerland, Durham, NC, USA (Nov 16–19, 2020). https://doi.org/10.1007/978-3-030-64378-2_10
8. Alwen, J., Jost, D., Mularczyk, M.: On the insider security of MLS. In: Dodis, Y., Shrimpton, T. (eds.) *Advances in Cryptology – CRYPTO 2022, Part II. Lecture Notes in Computer Science*, vol. 13508, pp. 34–68. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 15–18, 2022). https://doi.org/10.1007/978-3-031-15979-4_2
9. Alwen, J., Mularczyk, M., Tselekounis, Y.: Fork-resilient continuous group key agreement. In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology – CRYPTO 2023, Part IV. Lecture Notes in Computer Science*, vol. 14084, pp. 396–429. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 20–24, 2023). https://doi.org/10.1007/978-3-031-38551-3_13
10. Amine, O., Baghery, K., Pindado, Z., Ràfols, C.: Simulation extractable versions of groth’s zk-snark revisited. *Int. J. Inf. Secur.* **23**(1), 431–445 (Sep 2023). <https://doi.org/10.1007/s10207-023-00750-7>, <https://doi.org/10.1007/s10207-023-00750-7>
11. Banfi, F., Maurer, U.: Anonymous authenticated communication. In: Galdi, C., Jarecki, S. (eds.) *SCN 22: 13th International Conference on Security in Communication Networks. Lecture Notes in Computer Science*, vol. 13409, pp. 289–312. Springer, Cham, Switzerland, Amalfi, Italy (Sep 12–14, 2022). https://doi.org/10.1007/978-3-031-14791-3_13
12. Banfi, F., Maurer, U., Portmann, C., Zhu, J.: Composable and finite computational security of quantum message transmission. In: Hofheinz, D., Rosen, A. (eds.) *TCC 2019: 17th Theory of Cryptography Conference, Part I. Lecture*

- Notes in Computer Science, vol. 11891, pp. 282–311. Springer, Cham, Switzerland, Nuremberg, Germany (Dec 1–5, 2019). https://doi.org/10.1007/978-3-030-36030-6_12
13. Bender, A., Katz, J., Morselli, R.: Ring signatures: Stronger definitions, and constructions without random oracles. In: Halevi, S., Rabin, T. (eds.) TCC 2006: 3rd Theory of Cryptography Conference. Lecture Notes in Computer Science, vol. 3876, pp. 60–79. Springer Berlin Heidelberg, Germany, New York, NY, USA (Mar 4–7, 2006). https://doi.org/10.1007/11681878_4
 14. Bitansky, N., Canetti, R., Paneth, O., Rosen, A.: On the existence of extractable one-way functions. In: Shmoys, D.B. (ed.) 46th Annual ACM Symposium on Theory of Computing. pp. 505–514. ACM Press, New York, NY, USA (May 31 – Jun 3, 2014). <https://doi.org/10.1145/2591796.2591859>
 15. Boneh, D., Boyen, X.: Short signatures without random oracles. In: Cachin, C., Camenisch, J. (eds.) Advances in Cryptology – EUROCRYPT 2004. Lecture Notes in Computer Science, vol. 3027, pp. 56–73. Springer Berlin Heidelberg, Germany, Interlaken, Switzerland (May 2–6, 2004). https://doi.org/10.1007/978-3-540-24676-3_4
 16. Boneh, D., Lynn, B., Shacham, H.: Short signatures from the Weil pairing. In: Boyd, C. (ed.) Advances in Cryptology – ASIACRYPT 2001. Lecture Notes in Computer Science, vol. 2248, pp. 514–532. Springer Berlin Heidelberg, Germany, Gold Coast, Australia (Dec 9–13, 2001). https://doi.org/10.1007/3-540-45682-1_30
 17. Borisov, N., Goldberg, I., Brewer, E.A.: Off-the-record communication, or, why not to use PGP. In: Atluri, V., Syverson, P.F., di Vimercati, S.D.C. (eds.) WPES 2004. pp. 77–84. ACM (2004), <https://doi.org/10.1145/1029179.1029200>
 18. Chakraborty, S., Hofheinz, D., Maurer, U., Rito, G.: Deniable authentication when signing keys leak. In: Hazay, C., Stam, M. (eds.) Advances in Cryptology – EUROCRYPT 2023, Part III. Lecture Notes in Computer Science, vol. 14006, pp. 69–100. Springer, Cham, Switzerland, Lyon, France (Apr 23–27, 2023). https://doi.org/10.1007/978-3-031-30620-4_3
 19. Cramer, R., Damgård, I., Schoenmakers, B.: Proofs of partial knowledge and simplified design of witness hiding protocols. In: Desmedt, Y. (ed.) Advances in Cryptology – CRYPTO’94. Lecture Notes in Computer Science, vol. 839, pp. 174–187. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 21–25, 1994). https://doi.org/10.1007/3-540-48658-5_19
 20. Damgård, I., Haagh, H., Mercer, R., Nitulescu, A., Orlandi, C., Yakoubov, S.: Stronger security and constructions of multi-designated verifier signatures. In: Pass, R., Pietrzak, K. (eds.) TCC 2020: 18th Theory of Cryptography Conference, Part II. Lecture Notes in Computer Science, vol. 12551, pp. 229–260. Springer, Cham, Switzerland, Durham, NC, USA (Nov 16–19, 2020). https://doi.org/10.1007/978-3-030-64378-2_9
 21. Deignie, J., Duguey, C., Fouque, P.A.: MLS group messaging: How zero-knowledge can secure updates. In: Bertino, E., Shulman, H., Waidner, M. (eds.) ESORICS 2021: 26th European Symposium on Research in Computer Security, Part II. Lecture Notes in Computer Science, vol. 12973, pp. 587–607. Springer, Cham, Switzerland, Darmstadt, Germany (Oct 4–8, 2021). https://doi.org/10.1007/978-3-030-88428-4_29
 22. Diffie, W., Hellman, M.E.: New directions in cryptography. IEEE Transactions on Information Theory **22**(6), 644–654 (1976). <https://doi.org/10.1109/TIT.1976.1055638>

23. Dodis, Y., Katz, J., Smith, A., Walfish, S.: Composability and on-line deniability of authentication. In: Reingold, O. (ed.) TCC 2009: 6th Theory of Cryptography Conference. Lecture Notes in Computer Science, vol. 5444, pp. 146–162. Springer Berlin Heidelberg, Germany (Mar 15–17, 2009). https://doi.org/10.1007/978-3-642-00457-5_10
24. Fiedler, R., Langrehr, R.: On deniable authentication against malicious verifiers. In: Kalai, Y.T., Kamara, S.F. (eds.) Advances in Cryptology – CRYPTO 2025, Part VIII. Lecture Notes in Computer Science, vol. 16007, pp. 3–38. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 17–21, 2025). https://doi.org/10.1007/978-3-032-01913-4_1
25. Gajland, P., Janneck, J., Kiltz, E.: Ring signatures for deniable AKEM: Gandalf’s fellowship. In: Reyzin, L., Stebila, D. (eds.) Advances in Cryptology – CRYPTO 2024, Part I. Lecture Notes in Computer Science, vol. 14920, pp. 305–338. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 18–22, 2024). https://doi.org/10.1007/978-3-031-68376-3_10
26. Goldreich, O., Goldwasser, S., Micali, S.: How to construct random functions (extended abstract). In: 25th Annual Symposium on Foundations of Computer Science. pp. 464–479. IEEE Computer Society Press, Singer Island, Florida (Oct 24–26, 1984). <https://doi.org/10.1109/SFCS.1984.715949>
27. Groth, J.: Simulation-sound NIZK proofs for a practical language and constant size group signatures. In: Lai, X., Chen, K. (eds.) Advances in Cryptology – ASIACRYPT 2006. Lecture Notes in Computer Science, vol. 4284, pp. 444–459. Springer Berlin Heidelberg, Germany, Shanghai, China (Dec 3–7, 2006). https://doi.org/10.1007/11935230_29
28. Jakobsson, M., Sako, K., Impagliazzo, R.: Designated verifier proofs and their applications. In: Maurer, U.M. (ed.) Advances in Cryptology – EUROCRYPT’96. Lecture Notes in Computer Science, vol. 1070, pp. 143–154. Springer Berlin Heidelberg, Germany, Saragossa, Spain (May 12–16, 1996). https://doi.org/10.1007/3-540-68339-9_13
29. Janneck, J., Nishaburi, A., Rito, G.: Modeling emails: On the deniability of bccs. Cryptology ePrint Archive, Report 2025/1750 (2025), <https://eprint.iacr.org/2025/1750>
30. Jost, D., Maurer, U., Mularczyk, M.: Efficient ratcheting: Almost-optimal guarantees for secure messaging. In: Ishai, Y., Rijmen, V. (eds.) Advances in Cryptology – EUROCRYPT 2019, Part I. Lecture Notes in Computer Science, vol. 11476, pp. 159–188. Springer, Cham, Switzerland, Darmstadt, Germany (May 19–23, 2019). https://doi.org/10.1007/978-3-030-17653-2_6
31. Kiltz, E., Mityagin, A., Panjwani, S., Raghavan, B.: Append-only signatures. In: Caires, L., Italiano, G.F., Monteiro, L., Palamidessi, C., Yung, M. (eds.) ICALP 2005: 32nd International Colloquium on Automata, Languages and Programming. Lecture Notes in Computer Science, vol. 3580, pp. 434–445. Springer Berlin Heidelberg, Germany, Lisbon, Portugal (Jul 11–15, 2005). https://doi.org/10.1007/11523468_36
32. Klein, K., Pascual-Perez, G., Walter, M., Kamath, C., Capretto, M., Cueto, M., Markov, I., Yeo, M., Alwen, J., Pietrzak, K.: Keep the dirt: Tainted TreeKEM, adaptively and actively secure continuous group key agreement. In: 2021 IEEE Symposium on Security and Privacy. pp. 268–284. IEEE Computer Society Press, San Francisco, CA, USA (May 24–27, 2021). <https://doi.org/10.1109/SP40001.2021.00035>

33. Krawczyk, H., Rabin, T.: Chameleon signatures. In: ISOC Network and Distributed System Security Symposium – NDSS 2000. The Internet Society, San Diego, CA, USA (Feb 2–4, 2000)
34. Liu-Zhang, C.D., Portmann, C., Rito, G.: Stateful communication with malicious parties. Cryptology ePrint Archive, Report 2024/1593 (2024), <https://eprint.iacr.org/2024/1593>
35. Maurer, U., Portmann, C., Rito, G.: Giving an adversary guarantees (or: How to model designated verifier signatures in a composable framework). In: Tibouchi, M., Wang, H. (eds.) *Advances in Cryptology – ASIACRYPT 2021, Part III*. Lecture Notes in Computer Science, vol. 13092, pp. 189–219. Springer, Cham, Switzerland, Singapore (Dec 6–10, 2021). https://doi.org/10.1007/978-3-030-92078-4_7
36. Maurer, U., Portmann, C., Rito, G.: Multi-designated receiver signed public key encryption. In: Dunkelman, O., Dziembowski, S. (eds.) *Advances in Cryptology – EUROCRYPT 2022, Part II*. Lecture Notes in Computer Science, vol. 13276, pp. 644–673. Springer, Cham, Switzerland, Trondheim, Norway (May 30 – Jun 3, 2022). https://doi.org/10.1007/978-3-031-07085-3_22
37. Naor, M.: Deniable ring authentication. In: Yung, M. (ed.) *Advances in Cryptology – CRYPTO 2002*. Lecture Notes in Computer Science, vol. 2442, pp. 481–498. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 18–22, 2002). https://doi.org/10.1007/3-540-45708-9_31
38. Ristenpart, T., Yilek, S.: The power of proofs-of-possession: Securing multiparty signatures against rogue-key attacks. In: Naor, M. (ed.) *Advances in Cryptology – EUROCRYPT 2007*. Lecture Notes in Computer Science, vol. 4515, pp. 228–245. Springer Berlin Heidelberg, Germany, Barcelona, Spain (May 20–24, 2007). https://doi.org/10.1007/978-3-540-72540-4_13
39. Rivest, R.L., Shamir, A., Tauman, Y.: How to leak a secret. In: Boyd, C. (ed.) *Advances in Cryptology – ASIACRYPT 2001*. Lecture Notes in Computer Science, vol. 2248, pp. 552–565. Springer Berlin Heidelberg, Germany, Gold Coast, Australia (Dec 9–13, 2001). https://doi.org/10.1007/3-540-45682-1_32
40. Sahai, A.: Non-malleable non-interactive zero knowledge and adaptive chosen-ciphertext security. In: 40th Annual Symposium on Foundations of Computer Science. pp. 543–553. IEEE Computer Society Press, New York, NY, USA (Oct 17–19, 1999). <https://doi.org/10.1109/SFFCS.1999.814628>
41. Schnorr, C.P.: Efficient identification and signatures for smart cards. In: Brassard, G. (ed.) *Advances in Cryptology – CRYPTO’89*. Lecture Notes in Computer Science, vol. 435, pp. 239–252. Springer, New York, USA, Santa Barbara, CA, USA (Aug 20–24, 1990). https://doi.org/10.1007/0-387-34805-0_22
42. Secure Hash Standard (SHS). National Institute of Standards and Technology, NIST FIPS PUB 180-4, U.S. Department of Commerce (Aug 2015)
43. SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions. National Institute of Standards and Technology, NIST FIPS PUB 202, U.S. Department of Commerce (Aug 2015)
44. Shim, K.A.: Rogue-key attacks on the multi-designated verifiers signature scheme. *Information Processing Letters* **107**(2), 83–86 (2008). <https://doi.org/10.1016/j.ipl.2007.11.021>, <https://www.sciencedirect.com/science/article/pii/S0020019008000318>
45. Steinfeld, R., Bull, L., Wang, H., Pieprzyk, J.: Universal designated-verifier signatures. In: Lai, C.S. (ed.) *Advances in Cryptology – ASIACRYPT 2003*. Lecture Notes in Computer Science, vol. 2894, pp. 523–542. Springer Berlin Heidelberg,

- Germany, Taipei, Taiwan (Nov 30 – Dec 4, 2003). https://doi.org/10.1007/978-3-540-40061-5_33
46. Unger, N., Dechand, S., Bonneau, J., Fahl, S., Perl, H., Goldberg, I., Smith, M.: SoK: Secure messaging. In: 2015 IEEE Symposium on Security and Privacy. pp. 232–249. IEEE Computer Society Press, San Jose, CA, USA (May 17–21, 2015). <https://doi.org/10.1109/SP.2015.22>
 47. Weidner, M., Kleppmann, M., Hugenhroth, D., Beresford, A.R.: Key agreement for decentralized secure group messaging with strong security guarantees. In: Vigna, G., Shi, E. (eds.) ACM CCS 2021: 28th Conference on Computer and Communications Security. pp. 2024–2045. ACM Press, Virtual Event, Republic of Korea (Nov 15–19, 2021). <https://doi.org/10.1145/3460120.3484542>
 48. Zhang, Y., Au, M.H., Yang, G., Susilo, W.: (strong) multi-designated verifiers signatures secure against rogue key attack. In: Xu, L., Bertino, E., Mu, Y. (eds.) Network and System Security. pp. 334–347. Springer Berlin Heidelberg, Berlin, Heidelberg (2012)

Appendix

A Game-Based Security Definitions

A.1 One-Way Functions

A One-Way Function (OWF) with domain $\mathcal{X}$ and co-domain $\mathcal{Y}$ is a pair $\text{OWF} = (\text{Samp}, \text{F})$, where Samp and F are, respectively, probabilistic and deterministic algorithms. Samp is a sampler of $\mathcal{X}$, whereas F computes a function $f : \mathcal{X} \rightarrow \mathcal{Y}$; concretely:

- Samp : outputs a value $x \in \mathcal{X}$;
- $\text{F}(x \in \mathcal{X})$: outputs a value $y \in \mathcal{Y}$.

The One-Wayness notion (Definition 25) relies on oracles $\vec{\mathcal{O}}[\text{Samp}, \text{Submit}]$:

$\mathcal{O}[\text{Samp}](i \in \mathbb{N})$: If x_i is undefined then sample $x_i \leftarrow \text{Samp}$. Output $\text{F}(x_i)$.
 $\mathcal{O}[\text{Submit}](i \in \mathbb{N}, x)$: If x_i is undefined, query $\mathcal{O}[\text{Samp}](i)$. Output $\text{F}(x)$.

Definition 25 (One-Wayness: $\neg\text{OW}$). *A query $\mathcal{O}[\text{Samp}](i)$ outputs $y \in \mathcal{Y}$. A query $\mathcal{O}[\text{Submit}](i, x)$ also outputs y .*

An adversary $\mathbf{A}$ (ε, t) -breaks the (n) -One-Wayness of OWF if it runs in time t , queries oracles $\mathcal{O}[\text{Samp}]$ and $\mathcal{O}[\text{Submit}]$ on at most n different indices, and satisfies $\text{Adv}^{\text{OWF}}(\mathbf{A}) \geq \varepsilon$. We say OWF is (ε, t, n) -secure if there is no such adversary.

A.2 Hash Functions

A Hash Function (HF) with domain $\mathcal{X}$ and co-domain $\mathcal{Y}$ is a pair $\text{HF} = (\text{Setup}, \text{H})$ where Setup and H are, respectively, probabilistic and deterministic algorithms. Setup is a sampler of public parameters and H computes a function $f_{\text{pp}} : \mathcal{X} \rightarrow \mathcal{Y}$ (for given public parameters pp); concretely:

- Setup : outputs public parameters pp ;
- $\text{H}(\text{pp}, x \in \mathcal{X})$: outputs a value $y \in \mathcal{Y}$.

Often $\mathcal{X}$ is $\{0, 1\}^*$ (i.e. the input domain is the set of all finite-length bit strings) and $\mathcal{Y}$ is $\{0, 1\}^k$ (i.e. the co-domain is a set of bit strings with some fixed-length k). Hash Functions used in practice often consist only of algorithm H , i.e. algorithm Setup is not defined. In practice, one picks H from a set of standardized algorithms (e.g. one of the SHA3 functions). The Collision Resistance notion (Definition 26) relies on oracles $\vec{\mathcal{O}}[\text{Setup}, \text{Submit}]$:

$\mathcal{O}[\text{Setup}]$: If pp is undefined then sample $\text{pp} \leftarrow \text{Setup}$. Output pp .
 $\mathcal{O}[\text{Submit}](x \in \mathcal{X})$: $\text{pp} \leftarrow \mathcal{O}[\text{Setup}]$. Output $\text{H}(\text{pp}, x)$.

Definition 26 (Collision Resistance: $\neg\text{Coll-Res}$). *Query $\mathcal{O}[\text{Submit}](x)$ outputs y . Query $\mathcal{O}[\text{Submit}](x')$ with $x' \neq x$ also outputs y .*

An adversary $\mathbf{A}$ (ε, t) -breaks the (n) -Collision Resistance of HF if it runs in time t , queries $\mathcal{O}[\text{Submit}]$ on at most n different indices, and satisfies $\text{Adv}^{\text{HF}}(\mathbf{A}) \geq \varepsilon$. We say HF is (ε, t, n) -secure if there is no such adversary.

A.3 Public Key Encryption

A Public Key Encryption (PKE) is a triple $\text{PKE} = (\text{Gen}, \text{Enc}, \text{Dec})$ of probabilistic algorithms:

- Gen : outputs a key-pair (pk, sk) ;
- $\text{Enc}(\text{pk}, m)$: outputs a ciphertext c ;
- $\text{Dec}(\text{sk}, c)$: outputs a message m or an error symbol $\perp$.

Consider a $\text{PKE} = (\text{Gen}, \text{Enc}, \text{Dec})$. The security notions ahead rely on oracles $\vec{\mathcal{O}}[\text{SK}, \text{PK}, \text{Enc}, \text{Dec}]$:

$\mathcal{O}[\text{SK}](i \in \mathbb{N})$: if $(\text{pk}_i, \text{sk}_i)$ is undefined, sample $(\text{pk}_i, \text{sk}_i) \leftarrow \text{Gen}$; output $(\text{pk}_i, \text{sk}_i)$.
 $\mathcal{O}[\text{PK}](i \in \mathbb{N})$ if $(\text{pk}_i, \text{sk}_i)$ is undefined, sample $(\text{pk}_i, \text{sk}_i) \leftarrow \text{Gen}$; output pk_i ;
 $\mathcal{O}[\text{Enc}](i, m; r_{\S})$: $\text{pk}_i \leftarrow \mathcal{O}[\text{PK}](i)$; output $\text{Enc}_{\text{pk}_i}(m; r_{\S})$;
 $\mathcal{O}[\text{Dec}](i, c)$: $(\text{pk}_i, \text{sk}_i) \leftarrow \mathcal{O}[\text{SK}](i)$; output $\text{Dec}_{\text{sk}_i}(c)$.

Definition 27 (Correctness: $\neg \text{Corr}$). Query $\mathcal{O}[\text{Enc}](i, m; \cdot)$ outputs c ; query $\mathcal{O}[\text{Dec}](i, c)$ does not output m .

The notion below relies on the following oracle:

$\mathcal{O}[\text{Enc}^b](i, m_0, m_1)$: Output $\text{Enc}_{\text{pk}_i}(m_b)$.

Definition 28 (IND-CPA). $\vec{\mathcal{O}}_{\text{IND-CPA}}^b := \vec{\mathcal{O}}[\text{SK}, \text{PK}, \text{Enc}^b]$.

Disallowed: querying $\mathcal{O}[\text{SK}]$ and $\mathcal{O}[\text{Enc}^b]$ on the same index i .

An adversary $\mathbf{A}$ (ε)-breaks the (n, q_D) -Correctness of PKE if it queries $\mathcal{O}[\text{PK}]$, $\mathcal{O}[\text{SK}]$, $\mathcal{O}[\text{Enc}]$ and $\mathcal{O}[\text{Dec}]$ on at most n different indices, makes at most q_D queries to oracle $\mathcal{O}[\text{Dec}]$ and $\text{Adv}^{\neg \text{Corr}}(\mathbf{A}) \geq \varepsilon$. Similarly, an adversary $\mathbf{A}$ (ε, t)-breaks the (n, q_E) -IND-CPA security of PKE if it runs in time t , queries $\mathcal{O}[\text{PK}]$, $\mathcal{O}[\text{SK}]$ and $\mathcal{O}[\text{Enc}]$ on at most n different indices, makes at most q_E queries to oracle $\mathcal{O}[\text{Enc}]$ and $\text{Adv}^{\text{IND-CPA}}(\mathbf{A}) \geq \varepsilon$. Finally, we say PKE is

$$(\varepsilon_{\text{Corr}}, \varepsilon_{\text{IND-CPA}}, t, n, q_E, q_D)\text{-secure}$$

if no adversary $\mathbf{A}$:

1. $(\varepsilon_{\text{Corr}})$ -breaks PKE's (n, q_D) -Correctness, or
2. $(\varepsilon_{\text{IND-CPA}}, t)$ -breaks PKE's (n, q_E) -IND-CPA security.

A.4 Non-Interactive Proof System

Let R be some binary relation and L_R be its induced language. A *Non-Interactive Proof System* (NIPS) for L_R is a triple of algorithms $\text{NIPS} = (\text{Setup}, \text{Prv}, \text{Vrfy})$ where:

- Setup : outputs a common reference string crs ;
- $\text{Prv}_{\text{crs}}(x, w)$: given crs and a statement-witness pair $(x, w) \in R$, outputs a proof π ;

- $\text{Vrfy}_{\text{crs}}(x, \pi)$: given crs , statement x and proof π , outputs either 1 or 0 (respectively, accepts or rejects).

In the following definitions, consider a $\text{NIPS} = (\text{Setup}, \text{Prv}, \text{Vrfy})$ for a relation R . The security notions below (Definitions 29 and 30) provide adversaries with access to the following oracles:

$\mathcal{O}[\text{Setup}]$: If crs is undefined, compute and store $\text{crs} \leftarrow \text{Setup}$; output crs .
 $\mathcal{O}[\text{Prv}](x, w)$: Output $\text{Prv}_{\text{crs}}(x, w)$.
 $\mathcal{O}[\text{Vrfy}](x, \pi)$: Output $\text{Vrfy}_{\text{crs}}(x, \pi)$.

Definition 29 (Completeness: $\neg$ Complete). A query $\mathcal{O}[\text{Prv}](x, w)$ with $(x, w) \in R$ outputs π . A query $\mathcal{O}[\text{Vrfy}](x, \pi)$ outputs 0.

Definition 30 (Soundness: $\neg$ Sound). A query $\mathcal{O}[\text{Vrfy}](x \notin L_R, \pi)$ outputs 1.

We say that an adversary $\mathbf{A}$ (ε, t) -breaks the (q_P, q_V) -Completeness (resp. Soundness) of scheme NIPS if $\mathbf{A}$ runs in time at most t , makes at most q_P and q_V queries to oracles $\mathcal{O}[\text{Prv}]$ and $\mathcal{O}[\text{Vrfy}]$, respectively, and satisfies $\text{Adv}^{\text{Complete}}(\mathbf{A}) \geq \varepsilon$ (resp. $\text{Adv}^{\text{Sound}}(\mathbf{A}) \geq \varepsilon$). We say NIPS is $(\varepsilon_{\text{Complete}}, \varepsilon_{\text{Sound}}, t, q_P, q_V)$ -secure if no adversary $(\varepsilon_{\text{Complete}}, t)$ -breaks NIPS 's (q_P, q_V) -Completeness and no adversary $(\varepsilon_{\text{Sound}}, t)$ -breaks NIPS 's (q_P, q_V) -Soundness.

A.5 Non-Interactive Zero Knowledge

A Non-Interactive Zero Knowledge scheme $\text{NIZK} = (\text{Setup}, \text{Prv}, \text{Vrfy}, \text{Sim} = (\text{Sim.Setup}, \text{Sim.Prv}))$ for a relation R consists of a scheme $\text{NIPS} = (\text{Setup}, \text{Prv}, \text{Vrfy})$ for R and a simulator $\text{Sim} = (\text{Sim.Setup}, \text{Sim.Prv})$, where:

- Sim.Setup : outputs a Common Reference String/Trapdoor pair (crs, τ) ;
- $\text{Sim.Prv}(\text{crs}, \tau, x)$: given crs and τ as above, and given a statement x , outputs a proof π .

Consider a scheme $\text{NIZK} = (\text{Setup}, \text{Prv}, \text{Vrfy}, \text{Sim} = (\text{Sim.Setup}, \text{Sim.Prv}))$ for a relation R . We define the following oracles:

$\mathcal{O}[\text{Setup}^{\mathbf{b}}]$: If crs is undefined, then for $\mathbf{b} = \mathbf{0}$, sample $\text{crs} \leftarrow \text{Setup}$, and for $\mathbf{b} = \mathbf{1}$, sample $(\text{crs}, \tau) \leftarrow \text{Sim.Setup}$; in either case, output crs .
 $\mathcal{O}[\text{Prv}^{\mathbf{b}}](x, w)$: For $\mathbf{b} = \mathbf{0}$, compute and output $\text{Prv}_{\text{crs}}(x, w)$, and for $\mathbf{b} = \mathbf{1}$, compute and output $\text{Sim.Prv}_{(\text{crs}, \tau)}(x)$.

Definition 31 (Zero Knowledge). $\vec{\mathcal{O}}_{\text{ZK}}^{\mathbf{b}} := \vec{\mathcal{O}}[\text{Setup}^{\mathbf{b}}, \text{Prv}^{\mathbf{b}}, \text{Vrfy}]$.

Disallowed: queries to $\mathcal{O}[\text{Prv}^{\mathbf{b}}]$ on inputs $(x, w) \notin R$.

We now introduce Simulation Soundness for NIZK [40]. The game system defined by this notion provides adversaries with access to oracle $\mathcal{O}[\text{Setup}]$ defined above, to oracle $\mathcal{O}[\text{Vrfy}]$ and to the following $\mathcal{O}[\text{Prv}]$:

$\mathcal{O}[\text{Prv}](x)$: Compute and output $\text{Sim.Prv}_{(\text{crs}, \tau)}(x)$.

Definition 32 (Simulation Soundness: $\neg$ SS). A query $\mathcal{O}[\text{Vrfy}](x \notin L_R, \pi)$ outputs 1. No prior query $\mathcal{O}[\text{Prv}](x)$ output π .

Adversary $\mathbf{A}$ (ε, t) -breaks the (q_P, q_V) -Zero Knowledge (resp. Simulation Soundness) of scheme NIZK if it makes at most q_P and q_V queries to $\mathcal{O}[\text{Prv}]/\mathcal{O}[\text{Sim.Prv}]$ and $\mathcal{O}[\text{Vrfy}]$, respectively, and satisfies $\text{Adv}^{\text{ZK}}(\mathbf{A}) \geq \varepsilon$ (resp. $\text{Adv}^{\text{SS}}(\mathbf{A}) \geq \varepsilon$). Finally, NIZK is $(\varepsilon_{\text{Complete}}, \varepsilon_{\text{Sound}}, \varepsilon_{\text{ZK}}, \varepsilon_{\text{SS}}, t, q_P, q_V)$ -secure if NIPS (Setup, Prv, Vrfy) is $(\varepsilon_{\text{Complete}}, \varepsilon_{\text{Sound}}, t, q_P, q_V)$ -secure and no adversary $(\varepsilon_{\text{ZK}}, t)$ -breaks NIZK's (q_P, q_V) -Zero Knowledge, or $(\varepsilon_{\text{SS}}, t)$ -breaks NIZK's (q_P, q_V) -Simulation Soundness.

B Security Analysis of Ring Signature Construction

B.1 Correctness

Theorem 4. If NIZK is $(\varepsilon_{\text{Complete}}, \cdot, \cdot, \cdot, t, q_P, q_V)$ -secure then no adversary $\mathbf{A}$ $(\varepsilon_{\text{Complete}}, t)$ -breaks RSign's $(q_S := q_P, q_V)$ -Correctness with $t \approx t + t_{\text{Corr}}$, where t_{Corr} is the time to run RSign's $\mathbf{G}^{\text{Corr}}$ game (i.e. the oracles).

Proof. First, recall oracles $\vec{\mathcal{O}}[\text{Sign}, \text{Vrfy}]$:

$\mathcal{O}[\text{Sign}](i, \vec{V}, m)$: If $i \notin \vec{V}$, or if vk_i was defined by a query $\mathcal{O}[\text{Reg.RA}](i, \cdot)$, output $\perp$. If vk_i was defined by a query $\mathcal{O}[\text{Reg.RA}](i, \cdot)$, output $\perp$. For each $j \in \vec{V}$ such that vk_j is undefined, query $\mathcal{O}[\text{VK}](j)$. Output $\text{Sign}_{\text{pp}}(\text{sk}_i, \vec{v}, m)$;
 $\mathcal{O}[\text{Vrfy}](\vec{V}, m, \sigma)$: Output $\text{Vrfy}_{\text{pp}}(\vec{v}, m, \sigma)$.

Second, recall event $\neg\text{Corr}$ (Definition 7). Fix some $i \in \mathbb{N}$.

- The output of every query $\mathcal{O}[\text{Reg.RA}](i, \cdot)$ is of the form $(\cdot, \text{vk} = \perp, \cdot)$.
- A query $\mathcal{O}[\text{Sign}](i, \vec{V}, m)$ with $i \in \vec{V}$ outputs a signature $\sigma := (c, \pi)$.
- A later query $\mathcal{O}[\text{Vrfy}](\vec{V}, m, \sigma)$ outputs 0.

From this we know that if an adversary triggers event $\neg\text{Corr}$ then there is some index i for which $(\text{vk}_i, \text{sk}_i)$ was sampled using RSign.Gen, in particular the i that is fixed for event $\neg\text{Corr}$. From the definition of RSign.Gen, this key-pair includes a statement-witness pair (x, w) that was sampled using ROWR.Samp. Letting $R \subseteq \mathcal{X} \times \mathcal{W}$ be the relation of the underlying ROWR scheme, note that by definition any statement-witness pair (x, w) sampled using ROWR.Samp must be in the relation: $(x, w) \in R$.

Therefore, for any query $\mathcal{O}[\text{Sign}](i, \vec{V}, m)$ that is suitable to trigger event $\neg\text{Corr}$, RSign's algorithm Sign was run on input $(\text{sk}_i, \vec{v}, m)$. Now, note that we must have $\text{sk}_i.\text{vk} \in \vec{v}$ because otherwise the query would not be suitable (to trigger $\neg\text{Corr}$). Therefore Sign computes h, l, pad and c as described in the scheme. Letting r be the random coins used to create ciphertext c , note that this in particular implies $c := \text{CPA.Enc}(\text{pk}_{\text{CPA}}, (\text{sk}.w, \text{pad}, h); r)$. Finally, Sign computes a NIZK proof π attesting that $(\text{pk}_{\text{CPA}}, h, c, \vec{v})$ is in language L_{RS} , and it provides as witness the triple $(\text{sk}.w, \text{pad}, r)$. From the definition of R_{RS} , Equation 6.1,

we know that indeed $((\text{pk}_{\text{CPA}}, h, c, \vec{v}), (\text{sk}.w, \text{pad}, r))$ is in relation R_{RS} . The signature that **Sign** outputs is $\sigma := (c, \pi)$.

Now consider a query $\mathcal{O}[\text{Vrfy}](\vec{V}, m, \sigma)$. All this query does is running **RSig**'s verification — $\text{Vrfy}_{\text{pp}}(\vec{v}, m, \sigma)$ — and then outputting the bit resulting from this verification. In turn, $\text{Vrfy}_{\text{pp}}(\vec{v}, m, \sigma := (c, \pi))$ simply computes $h = \text{HF.H}(\vec{v}, m)$ and then verifies the validity of the NIZK proof π included in σ . The bit that **RSig.Vrfy** outputs — and therefore the output of oracle query $\mathcal{O}[\text{Vrfy}](\vec{V}, m, \sigma)$ — is the output of $\text{NIZK.Vrfy}((\text{pk}_{\text{CPA}}, h, c, \vec{v}) \in L_{\text{RS}}, \pi)$.

We conclude this proof by showing that if **RSig**'s event $\neg\text{Corr}$ occurred, then event $\neg\text{Complete}$ for **RSig**'s underlying NIZK occurred (see Definition 29):

- we already established that during signing, **RSig.Sign** obtained a statement-witness pair $((\text{pk}_{\text{CPA}}, h, c, \vec{v}), (\text{sk}.w, \text{pad}, r))$ that is in relation R_{RS} ;
- NIZK proof π in signature σ was generated using this statement-witness pair;
- from the determinism of **HF.H** we know that the hash h computed in verification matches the one computed in signing;
- pk_{CPA} is also the same for which the proof was created because it is part of the public parameters;
- c is part of the signature, and so it also matches the one that was in the statement for which π was computed;
- the definition of **RSig**'s event $\neg\text{Corr}$ requires that the vectors of parties $\vec{V}$ input to the $\mathcal{O}[\text{Sign}]$ and $\mathcal{O}[\text{Vrfy}]$ queries matches, and therefore the vector of public keys $\vec{v}$ used for signature verification also matches the one used to create π .

Altogether, this means that a trivial reduction to the Completeness of the underlying NIZK triggers event $\neg\text{Complete}$ if the adversary makes a pair of queries that triggers **RSig**'s event $\neg\text{Corr}$, concluding the proof. $\square$

B.2 Registration-Completeness

Theorem 5. *If there is no adversary that $(t^{\text{ROWR}}, \varepsilon)$ -breaks the $(n^{\text{ROWR}}, q_{\text{RA}}^{H, \text{ROWR}}, q_{\text{RA}}^{\text{ROWR}}, q_{\text{RU}}^{\text{ROWR}})$ -Registration-Completeness of ROWR, then there is no adversary that (ε, t) -breaks **RSig**'s $(n := n^{\text{ROWR}}, q_{\text{RA}}^H := q_{\text{RA}}^{H, \text{ROWR}}, q_{\text{RA}} := q_{\text{RA}}^{\text{ROWR}}, q_{\text{RU}} := q_{\text{RU}}^{\text{ROWR}})$ -Registration-Completeness with $t^{\text{ROWR}} \approx t + t_{\text{RegComplete}}$, where $t_{\text{RegComplete}}$ is the time to run **RSig**'s $\mathbf{G}^{\text{RegComplete}}$ game (i.e. the oracles).*

Proof. Follows trivially from the assumption on ROWR's security and the definitions of **RSig**'s key-pairs and **Reg** algorithms. $\square$

B.3 Unforgeability

Theorem 6. *If ROWR is*

$$(\cdot, \varepsilon_{\text{OW}}^{\text{ROWR}}, \varepsilon_{\text{WE}}^{\text{ROWR}}, \gamma_{\text{st-spread}}^{\text{ROWR}}, t^{\text{ROWR}}, n^{\text{ROWR}}, q_{\text{RA}}^{H, \text{ROWR}}, q_{\text{RA}}^{\text{ROWR}}, q_{\text{RU}}^{\text{ROWR}})\text{-secure, (B.1)}$$

NIZK is

$$(\cdot, \cdot, \varepsilon_{\text{ZK}}^{\text{NIZK}}, \varepsilon_{\text{SS}}^{\text{NIZK}}, t^{\text{NIZK}}, q_P^{\text{NIZK}}, q_V^{\text{NIZK}})\text{-secure}, \quad (\text{B.2})$$

PKE is

$$(\varepsilon_{\text{Corr}}^{\text{PKE}}, \varepsilon_{\text{CPA}}^{\text{PKE}}, t^{\text{PKE}}, 1, q_E^{\text{PKE}}, q_D^{\text{PKE}})\text{-secure} \quad (\text{B.3})$$

and HF is

$$(\varepsilon^{\text{HF}}, t^{\text{HF}}, n^{\text{HF}})\text{-secure} \quad (\text{B.4})$$

then no adversary $\mathbf{A}$ (ε, t) -breaks RSign's

$$\begin{aligned} (n := n^{\text{ROWR}}, \quad q_{\text{RA}}^H &:= q_{\text{RA}}^{H, \text{ROWR}}, \quad q_{\text{RA}} := q_{\text{RA}}^{\text{ROWR}}, \quad q_{\text{RU}} := q_{\text{RU}}^{\text{ROWR}}, \\ q_S &:= \min(q_P^{\text{NIZK}}, q_E^{\text{PKE}}), \cdot, \\ q_V &:= \min(q_V^{\text{NIZK}}, q_D^{\text{PKE}}), \cdot)\text{-Unforgeability}, \end{aligned}$$

with $\varepsilon > \varepsilon_{\text{ZK}}^{\text{NIZK}} + \varepsilon_{\text{SS}}^{\text{NIZK}} + \varepsilon_{\text{Corr}}^{\text{PKE}} + \varepsilon_{\text{CPA}}^{\text{PKE}} + \varepsilon_{\text{WE}}^{\text{ROWR}} + 2 \cdot \varepsilon_{\text{OW}}^{\text{ROWR}} + n \cdot 2^{-\gamma_{\text{st-spread}}^{\text{ROWR}}} + \varepsilon^{\text{HF}}$, and

$$t^{\text{NIZK}} \approx t + t_{\text{EUF-CMA}}$$

$$t^{\text{PKE}} \approx t + t_{\text{EUF-CMA}} + t_{\text{NIZK.Sim.Setup}} + q_S \cdot t_{\text{NIZK.Sim.Priv}}$$

$$t^{\text{HF}}, t^{\text{ROWR}} \approx t + t_{\text{EUF-CMA}} + t_{\text{NIZK.Sim.Setup}} + q_S \cdot t_{\text{NIZK.Sim.Priv}} + q_{\text{RA}} \cdot t_{\text{ROWR.Extract}}$$

where $t_{\text{EUF-CMA}}$ is the time to run RSign's $\mathbf{G}^{\text{EUF-CMA}}$ game (i.e. the oracles), $t_{\text{ROWR.Extract}}$ is the time to run ROWR's algorithm Extract, $t_{\text{NIZK.Sim.Setup}}$ is the time to run NIZK's Sim.Setup algorithm and $t_{\text{NIZK.Sim.Priv}}$ is the time to run NIZK's Sim.Priv algorithm.

Proof. We proceed by a sequence of hybrids.

$\mathbf{G}^{\text{EUF-CMA}} \rightsquigarrow \mathbf{G}^1$: Game $\mathbf{G}^1$ is just like the original game $\mathbf{G}^{\text{EUF-CMA}}$ except that now NIZK's **crs** and proofs are simulated (i.e. they are generated using NIZK's Sim.Setup and Sim.Priv algorithms. The winning condition for $\mathbf{G}^1$ is the same as for $\mathbf{G}^{\text{EUF-CMA}}$. A straightforward reduction to the Zero Knowledge of the underlying NIZK (Definition 31) allows to bound the difference in the winning probabilities by the distinguishing advantage of an adversary against NIZK's oracles $\tilde{\mathcal{O}}_{\text{ZK}}^{\mathbf{b}}$, for $\mathbf{b} \in \{0, 1\}$. On one hand the reduction uses NIZK's $\mathcal{O}[\text{Setup}^{\mathbf{b}}]$ to generate the **crs** that it includes in RSign's public parameters, and uses NIZK's $\mathcal{O}[\text{Priv}^{\mathbf{b}}]$ to generate the NIZK proofs that are part of signatures output by $\mathcal{O}[\text{Sign}]$ queries. Note that the NIZK proofs that are created during queries $\mathcal{O}[\text{Sign}]$ are always for statement-witness pairs $\left((\text{pk}_{\text{CPA}}, h, c, \vec{v}), (\text{sk}.w, \text{pad}, r) \right)$ that are in relation R_{RS} — so the reduction does not make any disallowed queries to NIZK's Zero Knowledge oracles. On the other hand, note that the reduction has access to all parties' secret keys, so it can handle any oracle queries. Note that the reduction queries the underlying NIZK's oracle $\mathcal{O}[\text{Priv}]$ only once for each $\mathcal{O}[\text{Sign}]$

query the adversary makes. Therefore, if $\mathbf{A}$ makes at most $q_S \leq q_P^{\text{NIZK}}$ queries to $\mathcal{O}[\text{Sign}]$, it follows from Equation B.2 that

$$\left| \Pr[\mathbf{AG}^1 \Rightarrow \neg\text{EUF-CMA}] - \Pr[\mathbf{AG}^{\text{EUF-CMA}} \Rightarrow \neg\text{EUF-CMA}] \right| \leq \varepsilon_{\text{ZK}}^{\text{NIZK}}.$$

(Note that the runtime of the reduction is bounded essentially by the time of adversary $\mathbf{A}$ plus the time to emulate $\mathbf{G}^{\text{EUF-CMA}}$.)

$\mathbf{G}^1 \rightsquigarrow \mathbf{G}^2$: Game $\mathbf{G}^2$ is just like $\mathbf{G}^1$ except that now $\mathcal{O}[\text{Sign}]$ queries are handled differently. Concretely, for queries $\mathcal{O}[\text{Sign}](i, \vec{V}, m)$, the CPA ciphertext c (which is part of the signature) no longer encrypts the witness of index i ($\mathbf{sk}_i.w$) together with (an appropriately long) padding pad and a hash h , and instead encrypts $0^{|\mathbf{sk}_i.w|}$ (in place of $\mathbf{sk}_i.w$) plus pad and h as before. The winning condition for $\mathbf{G}^2$ is the same as for $\mathbf{G}^1$. A straightforward reduction to the CPA security of the underlying Public Key Encryption scheme CPA allows to bound the difference in the winning probabilities by the distinguishing advantage of an adversary against CPA's oracles $\tilde{\mathcal{O}}_{\text{CPA}}^{\mathbf{b}}$, for $\mathbf{b} \in \{0, 1\}$. The reduction handles $\mathcal{O}[\text{Sign}]$ queries by using CPA's $\mathcal{O}[\text{Enc}^{\mathbf{b}}]$ to generate CPA ciphertexts as needed, and computes the NIZK proofs included in signatures by using NIZK's Sim.Priv algorithm — which does not require a witness to the statement being proven. Furthermore, note that the reduction has access to all parties' secret keys and that it never uses the decryption key corresponding to $\mathbf{pk}_{\text{CPA}}$, and therefore it can handle any oracle queries an adversary may make. Note that the reduction only queries for one public key to the CPA game of the Public Key Encryption and makes one query $\mathcal{O}[\text{Enc}^{\mathbf{b}}]$ per $\mathcal{O}[\text{Sign}]$ query that the adversary makes. Therefore, if adversary $\mathbf{A}$ makes at most $q_S \leq q_E^{\text{CPA}}$ queries to $\mathcal{O}[\text{Sign}]$, it follows from Equation B.3 that

$$\left| \Pr[\mathbf{AG}^2 \Rightarrow \neg\text{EUF-CMA}] - \Pr[\mathbf{AG}^1 \Rightarrow \neg\text{EUF-CMA}] \right| \leq \varepsilon_{\text{CPA}}^{\text{PKE}}.$$

(Note that the runtime of the reduction is bounded essentially by the time of adversary $\mathbf{A}$ together with the time to emulate $\mathbf{G}^2$, which in turn is the time to emulate $\mathbf{G}^{\text{EUF-CMA}}$ plus the time to generate one simulated crs using NIZK's Sim.Setup and up to q_S simulated proofs using NIZK's Sim.Priv .)

$\mathbf{G}^2 \rightsquigarrow \mathbf{G}^3$: Game $\mathbf{G}^3$ is just like $\mathbf{G}^2$ except that now for every verification key $\mathbf{vk}$ that is defined by a query to $\mathcal{O}[\text{Reg.RA}]$, game $\mathbf{G}^3$ runs the underlying ROWR's extraction algorithm (ROWR.Reg.Extract) to obtain a witness to $\mathbf{vk}$.¹⁰ If this extraction does not output a valid witness w for the verification key (i.e. one such that $(\mathbf{vk}, w) \in R$, where R is the underlying ROWR's relation) then the game aborts. The winning condition is the same as for the previous game. A straightforward reduction to breaking ROWR's Witness-Extractability (i.e. to trigger event $\neg\text{WE}$) allows to bound the difference between an adversary's winning probabilities against each of these games: in particular, note that a reduction has access to all parties' secret keys and the NIZK simulation trapdoor, so it can

¹⁰ Recall verification keys of our construction are statements of ROWR's relation.

handle any oracle queries. When the adversary makes a query $\mathcal{O}[\text{Reg.RA}](i, U)$, the reduction forwards this query to the underlying ROWR's oracle $\mathcal{O}[\text{Reg.RA}]$ (with the same input). Let $(\cdot, x, w, \cdot)$ be the output of this query. Note that a RSig verification key is only registered if this tuple is not $\perp$. After getting this quadruple back from the query to ROWR's oracle $\mathcal{O}[\text{Reg.RA}]$, the reduction makes query $\mathcal{O}[\in](i, w)$ to the underlying ROWR's oracle. Given this reduction, it is easy to see that if game $\mathbf{G}^3$ were to abort then ROWR's event $\neg\text{WE}$ would have occurred. Putting things together it follows

$$\left| \Pr[\mathbf{AG}^3 \Rightarrow \neg\text{EUF-CMA}] - \Pr[\mathbf{AG}^2 \Rightarrow \neg\text{EUF-CMA}] \right| \leq \varepsilon_{\text{WE}}^{\text{ROWR}}.$$

(As in the previous game hop, the runtime of the reduction is essentially the time of adversary $\mathbf{A}$ together with the time to emulate $\mathbf{G}^2$.)

$\mathbf{G}^3 \rightsquigarrow \mathbf{G}^4$: Game $\mathbf{G}^4$ is just like $\mathbf{G}^3$ except that now it is assumed that no verification key vk that is (successfully) registered by a query to $\mathcal{O}[\text{Reg.RA}]$ matches any verification key vk_i that was previously defined by a query to $\mathcal{O}[\text{VK}](i)$, unless there was a prior query $\mathcal{O}[\text{SK}](i)$. The winning condition is the same as for the previous game. One can bound the difference of an adversary's winning probabilities against the two games by the probability of triggering event $\neg\text{OW}$ to the underlying ROWR. Note that such reduction has to generate the simulated NIZK crs plus the corresponding proof simulation trapdoor and so it can simulate signature queries by honest signers without having access to their secret keys. The reduction can also handle all queries to $\vec{\mathcal{O}}[\text{SK}, \text{VK}]$ by relying on ROWR's underlying oracles — for indices i for which statement x_i was defined by a query $\mathcal{O}[\text{Reg.RA}](i, \cdot)$, the reduction simply outputs x_i — and can handle queries to $\vec{\mathcal{O}}[\text{Reg.RA}, \text{Reg.RU}]$ by emulating ROWR's respective oracles itself. If the adversary would make a query to $\mathcal{O}[\text{Reg.RA}]$ that would successfully register a verification key vk for which there is an index i such that verification key vk_i matches vk (i.e. $\text{vk}_i = \text{vk}$) and vk_i was defined by a prior query to $\mathcal{O}[\text{VK}]$, then the reduction would use the witness w it extracts from vk 's registration (using ROWR's Extract algorithm), and would make a query $\mathcal{O}[\in](i, w)$ to ROWR's underlying oracle. Recall we are assuming that the extraction of a valid witness (for vk) must succeed, or else game $\mathbf{G}^3$ will abort. Therefore, if the adversary were able to register through $\mathcal{O}[\text{Reg.RA}]$ a verification key vk that it had obtained through oracle $\mathcal{O}[\text{VK}]$, the reduction would recover a valid witness for this vk and trigger event $\neg\text{OW}$ thanks to the follow up query to ROWR's $\mathcal{O}[\in]$. It follows

$$\left| \Pr[\mathbf{AG}^4 \Rightarrow \neg\text{EUF-CMA}] - \Pr[\mathbf{AG}^3 \Rightarrow \neg\text{EUF-CMA}] \right| \leq \varepsilon_{\text{OW}}^{\text{ROWR}}.$$

(The runtime of the reduction is essentially the time of adversary $\mathbf{A}$ together with the time to emulate $\mathbf{G}^4$, which is essentially the time to emulate $\mathbf{G}^2$ plus the time to run ROWR's Reg.Extract algorithm to extract each verification key that the adversary successfully registers through oracle $\mathcal{O}[\text{Reg.RA}]$.)

$\mathbf{G}^4 \rightsquigarrow \mathbf{G}^5$: Game $\mathbf{G}^5$ is just like $\mathbf{G}^4$ except that now it is assumed that no two verification keys that were defined by a query to either $\vec{\mathcal{O}}[\text{SK}, \text{VK}]$ are equal. The winning condition is the same as for the previous game. From assumption Equation B.1 we know that ROWR is $\gamma_{\text{st-spread}}^{\text{ROWR}}$ -Statement-Spreadness (Definition 5). Each RSig verification key is simply one such statement; if an adversary's queries to oracles $\vec{\mathcal{O}}[\text{SK}, \text{VK}]$ only make at most n^{ROWR} verification keys with distinct indices to be defined, then a simple union bound then implies

$$\left| \Pr[\mathbf{AG}^5 \Rightarrow \neg\text{EUF-CMA}] - \Pr[\mathbf{AG}^4 \Rightarrow \neg\text{EUF-CMA}] \right| \leq n^{\text{ROWR}} \cdot 2^{-\gamma_{\text{st-spread}}^{\text{ROWR}}}.$$

$\mathbf{G}^5 \rightsquigarrow \mathbf{G}^6$: Game $\mathbf{G}^6$ is just like $\mathbf{G}^5$ except that now we make an extra assumption. Consider any query $\mathcal{O}[\text{Vrfy}](\vec{V}, m, \sigma := (c, \pi))$ that fulfills the conditions for event $\neg\text{EUF-CMA}$, and let $\vec{v}$ be $\vec{V}$'s corresponding public key vector and h be the output of $\text{HF.H}(\vec{v}, m)$. (Note that a query $\mathcal{O}[\text{Vrfy}](\vec{V}, m, \sigma := (c, \pi))$ can only fulfill the conditions for event $\neg\text{EUF-CMA}$ if it outputs 1, there was no query $\mathcal{O}[\text{Sign}](\cdot, \vec{V}, m)$ and there was no query $\mathcal{O}[\text{SK}](i \in \vec{V})$.) In $\mathbf{G}^6$, for any such query it is assumed that $(\text{pk}_{\text{CPA}}, h, c, \vec{v}) \in L_{\text{RS}}$. The winning condition for $\mathbf{G}^6$ is the same as $\mathbf{G}^5$'s.

One can bound the difference in the winning probabilities by a straightforward reduction to the Simulation Soundness of the underlying NIZK. The reduction generates (and therefore has access to) all parties' secret keys, so it can handle any oracle queries an adversary may make. The only thing the reduction does not have access to is the trapdoor τ for the NIZK; however it can still handle all necessary $\mathcal{O}[\text{Sign}]$ queries by using NIZK's $\mathcal{O}[\text{Prv}]$ to obtain NIZK proofs on false statements. The only case in which an adversary would win game $\mathbf{G}^5$ but would not win $\mathbf{G}^6$ is if it would make a RSig query $\mathcal{O}[\text{Vrfy}](\vec{V}, m, \sigma := (c, \pi))$ that would output 1 but yet:

- for $h \leftarrow \text{HF.H}(\vec{v}, m)$, the output of $\text{NIZK.Vrfy}(\text{crs}, (\text{pk}_{\text{CPA}}, h, c, \vec{v}) \in L_{\text{RS}}, \pi)$ in σ 's verification was 1 but yet $(\text{pk}_{\text{CPA}}, h, c, \vec{v}) \notin L_{\text{RS}}$;
- the adversary did not make a prior query $\mathcal{O}[\text{Sign}](i, \vec{V}, m)$; and
- the adversary did not make a prior query $\mathcal{O}[\text{SK}](i \in \vec{V})$.

Note that at this point we are making the following assumptions:

- all (honest) public keys are unique
- no verification key vk that is (successfully) registered by a query to $\mathcal{O}[\text{Reg.RA}]$ matches any verification key vk_i that was previously defined by a query to $\mathcal{O}[\text{VK}](i)$, unless there was a prior query $\mathcal{O}[\text{SK}](i)$.

Since we are assuming that the adversary did not make any prior query $\mathcal{O}[\text{SK}](i \in \vec{V})$, we know that no verification key matching one in $\vec{v}$ was registered through $\mathcal{O}[\text{Reg.RA}]$. Together, this implies that all prior $\mathcal{O}[\text{Sign}]$ queries proved NIZK statements that are different from the one that is verified during the $\mathcal{O}[\text{Vrfy}]$ query. Therefore, if $\mathcal{O}[\text{Vrfy}](\vec{V}, m, \sigma := (c, \pi))$ were to output 1, then event $\neg\text{SS}$ of the underlying NIZK's would have occurred because for this $\mathcal{O}[\text{Vrfy}]$ query

the reduction would have made a query $\mathcal{O}[\text{Vrfy}]((\text{pk}_{\text{CPA}}, h, c, \vec{v}), \pi)$ for which $(\text{pk}_{\text{CPA}}, h, c, \vec{v}) \notin L_{\text{RS}}$, and yet there was no prior query to $\mathcal{O}[\text{Prv}]$ on (false) statement $(\text{pk}_{\text{CPA}}, h, c, \vec{v}) \mathcal{O}[\text{Prv}](x)$ that had output π (since the reduction would only query $\mathcal{O}[\text{Prv}]$ upon a query $\mathcal{O}[\text{Sign}]$ by adversary). If adversary $\mathbf{A}$ makes at most $q_S \leq q_P^{\text{NIZK}}$ queries to $\mathcal{O}[\text{Sign}]$ and at most $q_V \leq q_V^{\text{NIZK}}$ queries to $\mathcal{O}[\text{Vrfy}]$, it follows from Equation B.1

$$\left| \Pr[\mathbf{AG}^6 \Rightarrow \neg\text{EUF-CMA}] - \Pr[\mathbf{AG}^5 \Rightarrow \neg\text{EUF-CMA}] \right| \leq \varepsilon_{\text{SS}}^{\text{NIZK}}.$$

(Note that the runtime of the reduction is bounded essentially by the time of adversary $\mathbf{A}$ together with the time to emulate $\mathbf{G}^5$, which in this case is the time to emulate $\mathbf{G}^{\text{EUF-CMA}}$ because the NIZK's simulated crs and proofs are obtained “for free” using queries to NIZK's Simulation Soundness oracles.)

$\mathbf{G}^6 \rightsquigarrow \mathbf{G}^7$: Game $\mathbf{G}^7$ is just like $\mathbf{G}^6$ except that now we make yet another assumption. Specifically, consider any every pair of queries $\mathcal{O}[\text{Sign}](\cdot, \vec{V}', m')$ and $\mathcal{O}[\text{Vrfy}](\vec{V}, m, \cdot)$ that the adversary makes for which $\mathcal{O}[\text{Sign}]$ output a non- $\perp$ value and the $\mathcal{O}[\text{Vrfy}]$ query fulfills the conditions for event $\neg\text{EUF-CMA}$. Then, if $(\vec{V}, m) \neq (\vec{V}', m')$, the h -component of the statement that is proved (using NIZK) in the $\mathcal{O}[\text{Sign}]$ query differs from the one that is verified in the $\mathcal{O}[\text{Vrfy}]$ query. (The winning condition is as before.)

One can bound the difference of an adversary's winning probabilities for $\mathbf{G}^6$ and $\mathbf{G}^7$ by the probability that one can break HF's Collision Resistance. To obtain this bound, we follow an argument analogous to the one used for game hop $\mathbf{G}^5 \rightsquigarrow \mathbf{G}^6$. First note that a RSig query $\mathcal{O}[\text{Vrfy}](\vec{V}, m, \sigma := (c, \pi))$ can only trigger event $\neg\text{EUF-CMA}$ if:

- for $h \leftarrow \text{HF.H}(\vec{v}, m)$, the output of $\text{NIZK.Vrfy}(\text{crs}, (\text{pk}_{\text{CPA}}, h, c, \vec{v}) \in L_{\text{RS}}, \pi)$ in σ 's verification was 1;
- the adversary did not make a prior query $\mathcal{O}[\text{Sign}](i, \vec{V}, m)$; and
- the adversary did not make a prior query $\mathcal{O}[\text{SK}](i \in \vec{V})$.

Now, at this point we are making the following assumptions:

- all (honest) public keys are unique
- no verification key vk that is (successfully) registered by a query to $\mathcal{O}[\text{Reg.RA}]$ matches any verification key vk_i that was previously defined by a query to $\mathcal{O}[\text{VK}](i)$, unless there was a prior query $\mathcal{O}[\text{SK}](i)$.

Since we are assuming that the adversary did not make any prior query $\mathcal{O}[\text{SK}](i \in \vec{V})$, we know that no verification key matching one in $\vec{v}$ was registered through $\mathcal{O}[\text{Reg.RA}]$. Together, this implies that if $\vec{V} \neq \vec{V}'$ then vectors' $\vec{V}$ and $\vec{V}'$'s corresponding public key vectors $\vec{v}$ and $\vec{v}'$ also differ — i.e. $\vec{v} \neq \vec{v}'$. Therefore, if $(\vec{V}, m) \neq (\vec{V}', m')$ then also $(\vec{v}, m) \neq (\vec{v}', m')$.

More concretely, note that a reduction to HF's Collision Resistance has access to all parties' secrets and therefore can handle any oracle queries. Such reduction would set the HF component of RSig's public parameters to be the output of HF's

oracle $\mathcal{O}[\text{Setup}]$, and would handle queries $\vec{\mathcal{O}}[\text{Sign}, \text{Vrfy}]$ as before, but would rely on HF's oracle $\mathcal{O}[\text{Submit}]$ to obtain the outputs of HF.H (instead of computing them directly). For any adversary $\mathbf{A}$ that only makes at most q_S and q_V queries to oracles $\mathcal{O}[\text{Sign}]$ and $\mathcal{O}[\text{Vrfy}]$, respectively, such that $q_S \cdot q_V \leq n^{\text{HF}}$, it follows from assumption Equation B.4:

$$\left| \Pr[\mathbf{AG}^7 \Rightarrow \neg\text{EUF-CMA}] - \Pr[\mathbf{AG}^6 \Rightarrow \neg\text{EUF-CMA}] \right| \leq \varepsilon^{\text{HF}}.$$

$\mathbf{G}^7 \rightsquigarrow \mathbf{G}^8$: Game $\mathbf{G}^8$ is just like $\mathbf{G}^7$ except that now we make an extra assumption. Consider any query $\mathcal{O}[\text{Vrfy}](\vec{V}, m, \sigma := (c, \pi))$ that fulfills the conditions for event $\neg\text{EUF-CMA}$. In $\mathbf{G}^8$ it is assumed that the decryption of c using the secret key sk_{CPA} — which is sampled together with the public parameter's pk_{CPA} — must result in a triple (w, pad, h) for which there are random coins r such that $((\text{pk}_{\text{CPA}}, h, c, \vec{v}), (w, \text{pad}, r)) \in R_{\text{RS}}$. (This means that w is the witness corresponding to one of the statements in $\vec{v}$.) ($\mathbf{G}^8$'s winning condition is the same as $\mathbf{G}^7$'s.)

Consider one such $\mathcal{O}[\text{Vrfy}](\vec{V}, m, \sigma := (c, \pi))$ query, i.e. one that fulfills the conditions for event $\neg\text{EUF-CMA}$. We know σ was not output by a query $\mathcal{O}[\text{Sign}](\cdot, \vec{V}, m)$ — on the same vector of parties and message — because otherwise the $\mathcal{O}[\text{Vrfy}]$ query could not be the cause for event $\neg\text{EUF-CMA}$. But in $\mathbf{G}^7$ we are assuming the Simulation Soundness of the NIZK, and furthermore we have already established that the $\mathcal{O}[\text{Vrfy}]$ query can only trigger event $\neg\text{EUF-CMA}$ if the statement that is verified was not proven by a query to $\mathcal{O}[\text{Sign}]$, and therefore $(\text{pk}_{\text{CPA}}, h, c, \vec{v}) \in L_{\text{RS}}$. In turn this implies, as explained above, that there is a triple (w, pad, r) such that $((\text{pk}_{\text{CPA}}, h, c, \vec{v}), (w, \text{pad}, r)) \in R_{\text{RS}}$.

Note that, a reduction to PKE's correctness has unbounded runtime, can make unbounded encryption queries, and can provide the randomness to be used for each of these encryption queries (see Definition 27). So to reduce to PKE's Correctness, we make the reduction obtain a PKE public key from a query $\mathcal{O}[\text{PK}](1)$, and then set the public parameter's public key pk_{CPA} to be the output of this query. Then, for each verification query $\mathcal{O}[\text{Vrfy}](\vec{V}, m, \sigma := (c, \pi))$ that fulfills the conditions for event $\neg\text{EUF-CMA}$, the reduction makes a series of encryption queries to the underlying PKE's encryption oracle. Concretely, for each triple (w, pad, r) , the reduction makes a query $\mathcal{O}[\text{Enc}](1, (w, \text{pad}, h); r)$. After all these encryption queries, the reduction makes one decryption query for the ciphertext that is in signature σ : $\mathcal{O}[\text{Dec}](1, c)$. It is easy to see that if the assumption made in $\mathbf{G}^8$ does not hold then event $\neg\text{Corr}$ for the underlying PKE scheme must have occurred. Therefore, from Equation B.3 it then follows:

$$\left| \Pr[\mathbf{AG}^8 \Rightarrow \neg\text{EUF-CMA}] - \Pr[\mathbf{AG}^7 \Rightarrow \neg\text{EUF-CMA}] \right| \leq \varepsilon_{\text{Corr}}^{\text{PKE}}.$$

Establishing the following claim concludes the proof.

Claim. $\Pr[\mathbf{AG}^8 \Rightarrow \neg\text{EUF-CMA}] \leq \varepsilon_{\text{OW}}^{\text{ROWR}}$.

Proof. Once again, a straightforward reduction to the security of ROWR allows us to bound the probability that an adversary $\mathbf{A}$ wins game $\mathbf{G}^8$. This is because a reduction to the One-Wayness of ROWR can handle all queries an adversary may make: $\mathcal{O}[\text{Sign}]$ queries can be handled without using the witnesses of any parties, and $\mathcal{O}[\text{SK}]$ queries can be handled by relying on the analogous ROWR oracle. Suppose adversary $\mathbf{A}$ makes a verification query $\mathcal{O}[\text{Vrfy}](\vec{V}, m, \sigma := (c, \pi))$ that fulfills the conditions for event $\neg\text{EUF-CMA}$. In this case the reduction simply uses the secret key sk_{CPA} that was sampled together with the public parameter's public key pk_{CPA} and uses it to decrypt the signature's ciphertext c . This decryption must output a valid witness for one of the statements in the vector of statements corresponding to the vector of parties $\vec{V}$. The reduction then uses the witness w that it obtains from this decryption and tests, for each statement in $\vec{v}$ if w is a valid witness for it. Following the arguments used for prior reductions, given we are assuming the $\mathcal{O}[\text{Vrfy}]$ query fulfills the conditions to trigger event $\neg\text{EUF-CMA}$ we know for some index $i \in \{1, \dots, |\vec{v}|\}$, w must be a valid witness for statement v_i . The reduction then makes a query to the underlying ROWR's oracle $\mathcal{O}[\in](V_i, w)$ and wins the game. Therefore, it follows from Equation B.1 that the advantage of adversary $\mathbf{A}$ can only be at most $\varepsilon_{\text{OW}}^{\text{ROWR}}$. $\square$

B.4 Anonymity

Theorem 7. *If ROWR is*

$$(\cdot, \cdot, \varepsilon_{\text{WE}}^{\text{ROWR}}, \cdot, t^{\text{ROWR}}, n^{\text{ROWR}}, q_{\text{RA}}^{H, \text{ROWR}}, q_{\text{RA}}^{\text{ROWR}}, q_{\text{RU}}^{\text{ROWR}})\text{-secure}, \quad (\text{B.5})$$

NIZK is

$$(\cdot, \cdot, \varepsilon_{\text{ZK}}^{\text{NIZK}}, \cdot, t^{\text{NIZK}}, q_P^{\text{NIZK}}, q_V^{\text{NIZK}})\text{-secure} \quad (\text{B.6})$$

and PKE is

$$(\cdot, \varepsilon_{\text{CPA}}^{\text{PKE}}, t^{\text{PKE}}, q_E^{\text{PKE}}, q_D^{\text{PKE}})\text{-secure} \quad (\text{B.7})$$

then no adversary $\mathbf{A}$ (ε, t) -breaks RSig's

$$(n := n^{\text{ROWR}}, q_{\text{RA}}^H := q_{\text{RA}}^{H, \text{ROWR}}, q_{\text{RA}} := q_{\text{RA}}^{\text{ROWR}}, q_{\text{RU}} := q_{\text{RU}}^{\text{ROWR}}, \\ q_S := \min(q_P^{\text{NIZK}}, q_E^{\text{PKE}}), \cdot, \cdot, \cdot)\text{-Anonymity},$$

with $\varepsilon > 2 \cdot (\varepsilon_{\text{WE}}^{\text{ROWR}} + \varepsilon_{\text{ZK}}^{\text{NIZK}} + \varepsilon_{\text{CPA}}^{\text{PKE}})$, and

$$t^{\text{ROWR}} \approx t + t_{\text{Anon}}$$

$$t^{\text{NIZK}} \approx t + t_{\text{Anon}} + q_{\text{RA}} \cdot t_{\text{ROWR.Extract}}$$

$$t^{\text{PKE}} \approx t + t_{\text{Anon}} + q_{\text{RA}} \cdot t_{\text{ROWR.Extract}} + t_{\text{NIZK.Sim.Setup}} + q_S \cdot t_{\text{NIZK.Sim.Priv}}$$

where t_{Anon} is the time to run RSig's $\mathbf{G}_{\mathbf{b}}^{\text{Anon}}$ games (i.e. the oracles), $t_{\text{ROWR.Extract}}$ is the time to run ROWR's algorithm Extract, $t_{\text{NIZK.Sim.Setup}}$ is the time to run NIZK's Sim.Setup algorithm and $t_{\text{NIZK.Sim.Priv}}$ is the time to run NIZK's Sim.Priv algorithm.

Proof. We proceed by a sequence of hybrids.

$\mathbf{G}_b^{\text{Anon}} \rightsquigarrow \mathbf{G}_b^1$: Game $\mathbf{G}_b^1$ is just like the original game $\mathbf{G}_b^{\text{Anon}}$ except that now for every verification key $\mathbf{vk}$ that is defined by a query to $\mathcal{O}[\text{Reg.RA}]$, game $\mathbf{G}_b^1$ runs the underlying ROWR's extraction algorithm (ROWR.Reg.Extract) to obtain a witness to $\mathbf{vk}$. If this extraction does not output a valid witness w for the verification key (i.e. one such that $(\mathbf{vk}, w) \in R$, where R is the underlying ROWR's relation) then the game aborts. The winning condition is the same as for the previous game. An argument analogous to the one used for game hop $\mathbf{G}^2 \rightsquigarrow \mathbf{G}^3$ of Theorem 6 then implies:

$$\left| \Pr[\mathbf{AG}_b^1 \Rightarrow \text{win}] - \Pr[\mathbf{AG}_b^{\text{Anon}} \Rightarrow \text{win}] \right| \leq \varepsilon_{\text{WE}}^{\text{ROWR}}.$$

(The runtime of the reduction is essentially the time of adversary $\mathbf{A}$ plus the time to emulate $\mathbf{G}_b^{\text{Anon}}$.)

$\mathbf{G}_b^1 \rightsquigarrow \mathbf{G}_b^2$: Game $\mathbf{G}_b^2$ is just like $\mathbf{G}_b^1$ except that now NIZK's crs and proofs are simulated (i.e. they are generated using NIZK's Sim.Setup and Sim.Prv algorithms. The winning condition for $\mathbf{G}_b^2$ is the same as for $\mathbf{G}_b^1$. A reduction to the Zero Knowledge of the underlying NIZK (Definition 31) allows to bound the difference in the winning probabilities by the distinguishing advantage of an adversary against NIZK's oracles $\tilde{\mathcal{O}}_{\text{ZK}}^b$, for $b \in \{0, 1\}$. (The details of the reduction are analogous to the ones used for game-hop $\mathbf{G}^{\text{EUF-CMA}} \rightsquigarrow \mathbf{G}^1$ in the proof of Theorem 6.) Therefore, if $\mathbf{A}$ makes at most $q_S \leq q_P^{\text{NIZK}}$ queries to $\mathcal{O}[\text{Sign}]$, it follows from Equation B.6 that

$$\left| \Pr[\mathbf{AG}_b^2 \Rightarrow \text{win}] - \Pr[\mathbf{AG}_b^1 \Rightarrow \text{win}] \right| \leq \varepsilon_{\text{ZK}}^{\text{NIZK}}.$$

(The reduction's runtime is bounded essentially by the runtime of adversary $\mathbf{A}$ added with the time to emulate $\mathbf{G}_b^{\text{Anon}}$ plus the time to extract the witnesses for each verification key $\mathbf{vk}$ that is defined via a query to oracle $\mathcal{O}[\text{Reg.RA}]$.)

$\mathbf{G}_b^2 \rightsquigarrow \mathbf{G}_b^3$: Game $\mathbf{G}_b^3$ is just like $\mathbf{G}_b^2$ except that now $\mathcal{O}[\text{Sign}]$ queries are handled differently. Concretely, for a query $\mathcal{O}[\text{Sign}](i, \vec{V}, m)$, the CPA ciphertext c (which is part of the signature) no longer encrypts the witness together with (an appropriately long) padding pad and a hash h , and instead encrypts $0^{|\text{sk}_i \cdot w|}$ plus pad and h as before. (Note that $\mathbf{G}_0^3$ and $\mathbf{G}_1^3$ encrypt different witnesses.) The winning condition for $\mathbf{G}_b^3$ is the same as for $\mathbf{G}_b^2$. This game hop is analogous to hop $\mathbf{G}^1 \rightsquigarrow \mathbf{G}^2$ in the proof of Theorem 6. Therefore, if adversary $\mathbf{A}$ makes at most $q_S \leq q_E^{\text{CPA}}$ queries to $\mathcal{O}[\text{Sign}]$, it follows from Equation B.7 that

$$\left| \Pr[\mathbf{AG}_b^3 \Rightarrow \text{win}] - \Pr[\mathbf{AG}_b^2 \Rightarrow \text{win}] \right| \leq \varepsilon_{\text{CPA}}^{\text{PKE}}.$$

(Note that the runtime of the reduction is bounded essentially by the time of adversary $\mathbf{A}$ together with the time to emulate $\mathbf{G}_b^2$.)

To conclude the proof, note that $\mathbf{G}_0^3$ and $\mathbf{G}_1^3$ have exactly the same behavior, i.e. they are perfectly indistinguishable. $\square$

C Proof of Theorem 2

Proof. First note that our construction is a Transitive Registrable One-Way Relation:

- our construction’s algorithms ($\text{Setup}, \text{Samp}, D_{\in R}, \text{Reg} := (\text{RU}, \text{RA}, \text{Extract})$) are a Registrable One-Way Relation for relation $R_{\mathcal{M}^*}^{\text{AOS}} := \bigcup_{s \in \mathcal{M}^*} R_s^{\text{AOS}}$. This is because, by assumption, the underlying ROWR-SR is a Registrable One-Way Relation with Simulatable Registration for relation $R_{\epsilon}^{\text{AOS}}$, and because our construction’s $D_{\in R}$ algorithm accepts every pair that ROWR-SR. $D_{\in R}$ accepts — i.e. the relation our TROWR decides contains the relation that the underlying ROWR-SR decides.
- The support of our construction’s **Samp** is a subset of relation $R_{\epsilon}^{\text{AOS}}$.
- Our construction’s **TrExtend.X** and **TrExtend.W** algorithms are deterministic. (The claim that **TrExtend.W** is deterministic is contingent on the underlying AOS’s algorithm **Append** being deterministic. Although, formally AOS’ **Append** algorithm is probabilistic, as explained in Remark 7 one can transform any AOS scheme with a probabilistic **Append** algorithm into one with a deterministic **Append** algorithm.)

Now we argue the security of our construction. First, note that Registration-Completeness follows trivially from the assumption that the underlying ROWR-SR is Registration-Complete. Second, our construction’s **TrExtend.W**-Associativity follows (also trivially) from the assumption that the underlying ROWR-SR’s relation is $R_{\epsilon}^{\text{AOS}}$ and that AOS is Perfectly Correct. Third, Transitive-Witness-Extractability follows (also trivially) from the assumption that the underlying ROWR-SR is Witness-Extractability plus the assumption that AOS is Perfectly Correct.

At this point it only remains to argue the Transitive-One-Way security of our construction. As a first step we hop to a hybrid that uses ROWR-SR’s simulated registration algorithm (Sim_{RU}) to handle queries to $\mathcal{O}[\text{Reg.RU}]$. This allows us to handle $\mathcal{O}[\text{Reg.RU}]$ without knowledge of ROWR-SR’s witnesses (which are the random coins input to AOS’ **Setup** algorithm). After noticing this, the security proof becomes (essentially) a straightforward argument to breaking the Unforgeability of the underlying AOS. Concretely, it first guesses the index i for which the adversary will make a query $\mathcal{O}[\epsilon](i, e, w)$ that triggers event $\neg\text{TOW}$, and for this index it sets the ROWR-SR statement to be the verification key output by the underlying AOS Unforgeability game. If the adversary manages to trigger event $\neg\text{TOW}$ and the reduction guesses the correct index i (for the query $\mathcal{O}[\epsilon](i, e, w)$ the adversary makes that triggers event $\neg\text{TOW}$) then the reduction also triggers event $\neg\text{Prefix-EUF-CMA}$ of the underlying AOS. The final step is just noting that the probability the reduction guesses the correct index is at least $1/n$, where n is an upper bound on the number of difference indices that the TROWR adversary queries the TROWR’s oracles on. $\square$

D Proofs of One-Way Relation Facts

D.1 Proof of Proposition 1

Proof. This result follows from the security analysis of transformations R — which constructs a One-Way Relation from a One-Way Function — and F — vice-versa — defined below.

Consider a One-Way Relation $OWR = (\text{Samp}, D_{\in R})$ for a relation $R \subseteq \mathcal{X} \times \mathcal{W}$ and let $\mathcal{R}_{OWF}$ be the randomness space of $OWR.\text{Samp}$. We construct a One-Way Function $F[OWR] := (\text{Samp}_F, F)$ with domain $\mathcal{R}_{OWF}$ and co-domain $\mathcal{X}$:

Samp_F : Sample $r \leftarrow \mathcal{R}_{OWF}$; output r .

$F(r \in \mathcal{R}_{OWF})$: Compute $(x, \cdot) \leftarrow OWR.\text{Samp}(r)$; output x .

Consider a One-Way Function $OWF = (\text{Samp}, F)$ with domain $\mathcal{X}$ and co-domain $\mathcal{Y}$. We construct a One-Way Relation $R[OWF] := (\text{Samp}_R, D_{\in R})$ for relation $R := \{(x, y) \in \mathcal{X} \times \mathcal{Y} \mid F(x) = y\} \subseteq \mathcal{X} \times \mathcal{Y}$:

Samp_R : Sample $x \leftarrow OWF.\text{Samp}$; output $(OWF.F(x), x)$.

$D_{\in R}(x, w)$: Output $x \stackrel{?}{=} OWF.F(w)$. □

D.2 Proof of Proposition 2

Proof. This follows from the analysis of transformation R^{adap} given ahead. Consider a One-Way Relation $OWR = (\text{Samp}, D_{\in R})$ for a relation $R \subseteq \mathcal{X} \times \mathcal{W}$. We construct a One-Way Relation $R^{\text{adap}}[OWR] := (\text{Samp}_R^{\text{adap}}, D_{\in R}^{\text{adap}})$ for relation $R^{\text{adap}} := \{((x_0, x_1), w) \in \mathcal{X}^2 \times \mathcal{W} \mid (x_0, w) \in R \vee (x_1, w) \in R\} \subseteq \mathcal{X}^2 \times \mathcal{W}$:

$\text{Samp}_R^{\text{adap}}$: Sample $b \leftarrow \{0, 1\}$, and $(x_i, w_i) \leftarrow \text{Samp}$, for $i \in \{0, 1\}$; output $((x_0, x_1), w_b)$.

$D_{\in R}^{\text{adap}}((x_0, x_1), w)$: Output $(x_0, w) \in R \vee (x_1, w) \in R$. □

E Attacks on Plausible Deniability in Weaker Setups

There are various ways one can mount attacks on schemes providing deniable authentication [28, 44, 48, 23, 20]. To argue why the setup we introduce seems necessary to enable such guarantees, we now describe a sequence of attacks that break plausible deniability guarantees due to too weak setup assumptions.

Setting: Consider an honest Alice, a dishonest Bob and a dishonest Judy. Alice knows Bob is dishonest, but she still needs to send him an authenticated message m . However, she needs that Bob, and only Bob, can actually tell she sent it. In particular, Bob should not be able to convince Judy that Alice sent him m . Since Alice knows Bob is dishonest and will try convincing Judy of whatever messages she sends him, she decides to use a Ring Signature scheme $\text{RSig} = (\text{Gen}, \text{Sign}, \text{Vrfy})$ to authenticate her messages to Bob, with herself and Bob as the ring members, i.e. $\vec{V} = (A, B)$.

We begin by considering the case where Bob convinces Judy that Alice signed message m by proving to Judy that he could not have signed m with respect to his public key.

Secret key oblivious sampling of public keys: Suppose Bob had already sent Alice his public key, $\mathbf{pk}_B$, which Alice now uses to include him in the ring (i.e. which Alice will input to $\mathbf{RSig}$'s signing algorithm, $\mathbf{Sign}$). Alice runs $\mathbf{RSig.Sign}$ with her secret key $\mathbf{sk}_A$, Bob's public key $\mathbf{pk}_B$ and message m as input, which then results in a signature σ_m that she sends to Bob. When Bob receives σ_m , he verifies it and gets convinced Alice sent it. However, he also immediately sends it to Judy. Not only that, but he also proves to Judy that his public key $\mathbf{pk}_B$ was sampled in a way such that it is plausible Bob does not actually know the corresponding secret key, which implies he does not know how to use it to sign any messages [28, Section 7, Point 1 of the first enumerate.]. *Example [48]:* For a scheme where public keys are elements of some group $\mathbb{G}$ with generator $\langle g \rangle = \mathbb{G}$ fixed by the scheme, and where signing requires knowing the discrete logarithm of a party's public key, if Bob proves to Judy that $\mathbf{pk}_B$ is actually the output of a given hash function (e.g. SHA3) on some arbitrary input string, then it is plausible that Bob does not know $\mathbf{pk}_B$'s discrete logarithm with respect to g . In turn, this would mean that Bob could not have generated σ_m . So, Judy gets convinced that Bob did not produce σ_m ; therefore, Alice must have been the one signing m .

Enforcing public key sampling is not secret key oblivious: Ensuring that Bob's public key $\mathbf{pk}_B$ is sampled using the prescribed key-generation algorithm is relatively straightforward — for instance it can be achieved via a Non-Interactive Zero Knowledge proof. However, this is still not sufficient to ensure Bob knows the secret key.

Secret key obliviousness: Another approach Bob can follow is to ask Judy to generate his $(\mathbf{pk}_B, \mathbf{sk}_B)$ key-pair on his behalf, and only give him the public component $\mathbf{pk}_B$ — which he sends to Alice as being his own [28, Section 7, Point 2 of the first enumerate.]. In this case, Judy will also know that Bob could not have feasibly forged a signature because this would contradict $\mathbf{RSig}$'s unforgeability. In turn, this would mean that Judy would once again learn that Alice was the actual signer.

Guaranteeing signing capabilities: All these attacks boil down to Bob proving to Judy that he does not have the capability of signing; to prevent them it suffices to guarantee that dishonest Bob has the ability to sign with respect to $\mathbf{pk}_B$. In fact, since plausible deniability is a guarantee given to Alice, it is crucial that she is convinced that Bob can sign. This could be achieved either by having Bob prove his capability directly to Alice — e.g. by interacting directly with her — or by relying on a third party which would function as a Registration Authority — to whom Bob would prove his capabilities — on which Alice trusts. Naturally, if Bob proves to Alice that he can sign with respect to $\mathbf{pk}_B$, then he cannot later prove to Judy otherwise.

E.1 Anonymity Requirements

Another way Bob can break plausible deniability is by ensuring that Judy can identify the sender of each signature [20, Remark 1].

Breaking anonymity: If Bob were already expecting Alice to send a signature to him, he could simply tell Judy to intersect any messages that Alice sends to make sure Bob did not create them. For example, Judy could record all messages coming out of Alice's computer. In this case, if Judy sees a signature by Alice that verifies as valid with respect to ring $\vec{V} = (A, B)$, then she gets convinced that Alice actually did sign m .

Fixing anonymity: The only way to prevent this attack is by assuming a sender-anonymous channel to which both Alice and Bob can write, i.e. such that Judy cannot tell who (among Alice and Bob) actually wrote each transmitted message. This matches the type of channel that the composable treatments of Ring Signatures, MDVS and MDRS-PKE schemes from [11, 35, 34] assume.